cspm-lite

Construire un scanner de posture de securite cloud AWS, et comprendre pourquoi chaque brique existe

Guide de projet complet : architecture, roadmap en 10 phases, backlog de 30 issues, fiches de concepts, valorisation professionnelle.

Public : etudiant en genie informatique oriente cybersecurite, cloud et data.

Effort estime : 60 a 90 heures, soit 10 a 12 semaines a raison de 6 a 8 h par semaine.

Genere le 2026-07-29.

cspm-lite - guide d'apprentissage

Table des matieres

Guide d'apprentissage — cspm-lite 4
- Contenu 4
- Par où commencer 4
- Régénérer le PDF 4
- Trois principes pour ne pas te perdre 4

1. Le projet : cspm-lite 6
- 1.1 En une phrase 6
- 1.2 C'est quoi un CSPM ? 6
- 1.3 Pourquoi ce projet est un excellent choix pour toi 6
- 1.4 Architecture 6
- 1.5 Les composants, fichier par fichier 7
- 1.6 Glossaire (à connaître par cœur pour un entretien) 8
- 1.7 Règles de sécurité personnelles pour ce projet 8

2. Roadmap : les 10 phases 9
- Phase 0 — Fondations du projet 9
- Phase 1 — Le modèle de données et le moteur 9
- Phase 2 — Authentification et collecteurs 10
- Phase 3 — Les checks de sécurité (le cœur du projet) 11
- Phase 4 — Les reporters 12
- Phase 5 — La CLI 13
- Phase 6 — Le labo Terraform 13
- Phase 7 — Tests et qualité 14
- Phase 8 — CI/CD, Docker et supply chain 14
- Phase 9 — Le volet data 15
- Phase 10 — Finition et valorisation 15
- Récapitulatif 16

3. Backlog : 30 issues prêtes à créer 17
- Comment travailler ces issues 19

4. Fiches de concepts clés 20
- 4.1 Le modèle de responsabilité partagée 20
- 4.2 IAM : les 6 notions à maîtriser 20
- 4.3 S3 : pourquoi les buckets fuient 20
- 4.4 Réseau : security groups et NACL 21
- 4.5 IMDS et la brèche Capital One 21
- 4.6 CloudTrail : sans logs, pas d'investigation 21
- 4.7 Le CIS AWS Foundations Benchmark 22
- 4.8 ASFF : parler la langue de l'écosystème 22
- 4.9 boto3 : les pièges de l'API AWS 22
- 4.10 moto : simuler AWS en local 22
- 4.11 Terraform : le vocabulaire minimum 23
- 4.12 Docker et supply chain 23

5. Compétences acquises et valorisation 24
- 5.1 Ce que ce projet met réellement sur ton CV 24
- 5.2 Formulations pour ton CV 24
- 5.3 Les questions d'entretien que ce projet te prépare à gérer 24
- 5.4 Certifications cohérentes avec ce projet 25
- 5.5 Extensions possibles (après la v1) 25
- 5.6 Le plan de travail réaliste 25

6. Journal de bord et suivi 27
- 6.1 Pourquoi tenir un journal 27

6.2 Modèle d'entrée 27
6.3 Tableau de progression 27
6.4 Auto-évaluation : sais-tu expliquer… ? 29
6.5 Ressources de référence 29
7. Maximiser le bagage 31
7.1 Les quatre leviers 31
7.2 L'extra mile, phase par phase 31
7.3 Les trois pistes parallèles 32
7.4 Contribuer à un vrai projet open source 33
7.5 Les quatre extensions multiplicatrices 33
7.6 Retenir ce que tu apprends 34
7.7 Le carnet de preuves 34
7.8 Ce qui détruit le bagage 34
7.9 Après cspm-lite : les 12 mois suivants 35
7.10 Checklist finale : ai-je vraiment capitalisé ? 35
8. Les exercices — protocole étape par étape 36
Les six exercices 36
Le protocole, étape par étape 36
Règles 37
Que faire quand un test échoue 38
8. Présenter le projet en entretien 39
8.1 La règle d'or 39
8.2 Le pitch de 30 secondes 39
8.3 La version 2 minutes 39
8.4 Le tableau blanc, en 90 secondes 39
8.5 Les cinq histoires à avoir prêtes 40
8.6 Les questions de relance et comment y répondre 40
8.7 Adapter selon l'interlocuteur 41
8.8 Les pièges à éviter 41
8.9 La veille de l'entretien 41

Guide d'apprentissage — cspm-lite

Ce dossier est ton compagnon de route pour construire **cspm-lite** en comprenant chaque étape, pas en la copiant.

Contenu

Fichier	Contenu
01-le-projet.md	Ce qu'est un CSPM, l'architecture, le glossaire, les règles de sécurité du projet
02-roadmap.md	Les 10 phases : étapes, ce que tu apprends, définition de « fini »
03-issues.md	30 issues GitHub prêtes à créer, avec labels et DoD
04-concepts-cles.md	Les fiches à savoir expliquer sans notes
05-competences-et-valorisation.md	CV, entretien, certifications, extensions, planning 12 semaines
06-journal-de-bord.md	Modèle de journal, checklist de progression, ressources
07-maximiser-le-bagage.md	Comment convertir 80 h de travail en compétence durable et démontrable
exercices/	**6 exercices à coder toi-même**, 153 tests automatiques, protocole de correction
08-pitch-entretien.md	Pitchs 30 s / 2 min, tableau blanc, 5 histoires, questions de relance
scripts/create_issues.sh	Crée les labels et les 30 issues via gh
tools/build_pdf.py	Génère le PDF complet du guide
`cspm-lite-guide.pdf`	Le guide entier en un seul PDF

Par où commencer

1. Lis 01-le-projet.md en entier (15 min).
2. Survole 02-roadmap.md pour avoir la carte, sans tout retenir.
3. Lis 07-maximiser-le-bagage.md **avant** de commencer à coder : les pistes parallèles se mènent pendant le projet, pas après.
4. Crée les issues : `bash learning/scripts/create_issues.sh` (nécessite gh).
5. Attaque la phase 0. Une issue, une branche, une PR.
6. **Fais les exercices** au fur et à mesure des phases correspondantes : `bash learning/exercices/verifier.sh` pour voir où tu en es. Demande-moi « corrige mon ex1 » quand les tests sont verts.
7. Après chaque session, remplis une entrée de journal.

Régénérer le PDF

```
python3 learning/tools/build_pdf.py
```

Aucune dépendance externe : le script écrit le PDF directement.

Trois principes pour ne pas te perdre

Comprendre avant d'écrire. Pour chaque check, tu dois pouvoir dire ce qu'un attaquant ferait de la

misconfiguration. Si tu ne peux pas, ne code pas le check encore — lis d'abord.

Fini vaut mieux que parfait. Ne repars pas améliorer la phase 2 alors que tu es en phase 5. La phase 10 est ce qui donne sa valeur au projet.

Rien de sensible dans Git. Pas de clé, pas de `.tfstate`, pas de capture d'écran avec un ARN de compte réel. Vérifie avec `git diff --cached` avant chaque commit.

1. Le projet : cspm-lite

1.1 En une phrase

cspm-lite est un scanner de posture de sécurité cloud (CSPM) pour AWS, écrit en Python : il se connecte en lecture seule à un compte AWS, inventorie les ressources, évalue des dizaines de règles de sécurité, et produit des rapports exploitables (JSON, Markdown, HTML, ASFF pour AWS Security Hub).

1.2 C'est quoi un CSPM ?

CSPM = *Cloud Security Posture Management*.

Dans le cloud, la majorité des incidents ne viennent pas d'un exploit sophistiqué, mais d'une **mauvaise configuration** : un bucket S3 laissé public, une clé d'accès IAM qui traîne depuis 3 ans, un port 22 ouvert sur `0.0.0.0/0`, CloudTrail désactivé dans une région oubliée. C'est le modèle de **responsabilité partagée** : AWS sécurise le cloud, toi tu sécurises *dans* le cloud.

Un CSPM automatise la question : *« est-ce que mon compte cloud est configuré correctement, en continu, selon un référentiel reconnu ? »*

Les outils réels de ce marché :

Outil	Type	Remarque
Prowler	Open source, Python	La référence. ~500 checks AWS.
Scout Suite	Open source, Python	Sortie HTML, NCC Group.
Steampipe / Powerpipe	Open source, Go + SQL	Le cloud vu comme une base SQL.
AWS Security Hub	Managed AWS	Agrège les findings au format ASFF.
Wiz / Orca / Prisma Cloud	Commercial	CNAPP, agentless, très cher.

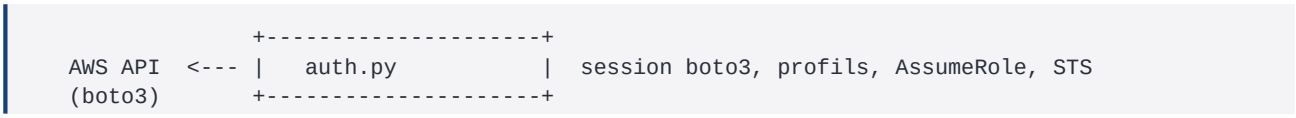
Tu construis une version pédagogique mais **architecturalement honnête** de Prowler. C'est exactement pour ça que le projet a de la valeur : tu ne réimplémentes pas un TP, tu reconstruis un outil dont l'industrie se sert.

1.3 Pourquoi ce projet est un excellent choix pour toi

Tu es étudiant en génie informatique, intéressé par **cybersécurité + cloud + data**. Ce projet est l'un des rares qui touche les trois axes en même temps, sans être artificiel :

- **Cybersécurité** — tu implémentes les contrôles du CIS AWS Foundations Benchmark. Tu apprends *pourquoi* chaque misconfiguration est dangereuse, pas juste sa détection. C'est du contenu d'entretien direct.
- **Cloud** — tu manipules IAM, S3, EC2/VPC, CloudTrail, KMS, STS via l'API. Tu provisionnes ton propre labo en Terraform. Tu comprends la pagination, le throttling, le multi-région, l'`AssumeRole`.
- **Data** — tu modélises un schéma de findings, tu normalises des sources hétérogènes, tu agrèges, tu scores, tu historises et tu visualises la dérive de posture dans le temps. C'est un vrai mini-pipeline ETL.
- **Génie logiciel** — packaging Python, pattern registry/plugins, typage, tests avec mocks, CI/CD, Docker, sécurité de la supply chain.

1.4 Architecture



```
                    |
                    v
              +---------------------+
              |   collectors/       |  scan_s3, scan_iam, scan_ec2, scan_cloudtrail
              |   (EXTRACT)         |  -> pagination, multi-region, parallelisme
              +---------------------+
                    |
                    v  liste de Resource (modele normalise)
              +---------------------+
              |   engine/models.py  |  Resource, Finding, Severity, ScanResult
              |   (TRANSFORM)       |
              +---------------------+
                    |
                    v
              +---------------------+
              |   engine/registry   |  registre de checks (decorateur @check)
              |   engine/checks/    |  s3_checks, iam_checks, net_checks, log_checks
              |   (RULES)           |  -> mappes sur CIS AWS Benchmark
              +---------------------+
                    |
                    v  liste de Finding
              +---------------------+
              |   reporters/        |  json / markdown / html / asff
              |   (LOAD)            |
              +---------------------+
                    ^
                    |
              +---------------------+
              |   cli.py            |  Typer + Rich, codes de sortie pour la CI
              +---------------------+

lab/          Terraform : un compte AWS volontairement vulnerable pour tester
tests/        pytest + moto : AWS simule, zero cout, zero risque
.github/      CI : lint, types, tests, build image, scan de l'image
Dockerfile    image multi-stage, non-root
```

Le point clé de ce design : **collecte et évaluation sont séparées**. Les collectors ne savent rien de la sécurité, les checks ne savent rien de boto3. C'est ce qui rend l'outil testable (on injecte de faux inventaires) et extensible (ajouter un check = ajouter une fonction, pas toucher au reste).

1.5 Les composants, fichier par fichier

Fichier	Rôle	Concept d'ingénierie
`src/cspm_lite/auth.py`	Créer une session AWS (profil, rôle, région)	Chaîne de credentials, STS, moindre privilège
`collectors/scan_s3.py`	Lister buckets + ACL + policy + chiffrement	Pagination, appels par ressource, gestion 404
`collectors/scan_iam.py`	Users, clés, policies, credential report	API asynchrone (rapport à générer puis lire)
`collectors/scan_ec2.py`	Security groups, instances, EBS, VPC	Multi-région, parallélisme
`collectors/scan_cloudtrail.py`	Trails, config, statut	Détection de la couverture d'audit
`engine/models.py`	`Resource`, `Finding`, `Severity`, `ScanResult`	Modélisation de données, typage, sérialisation
`engine/registry.py`	Registre de checks via décorateur	Pattern plugin / inversion de contrôle
`engine/checks/*.py`	Les règles de sécurité	Référentiel CIS, logique métier pure
`reporters/*.py`	Sorties JSON / MD / HTML / ASFF	Formats standards, dataviz
`cli.py`	Interface ligne de commande	UX développeur, exit codes, config
`lab/*.tf`	Labo AWS vulnérable	Infrastructure as Code

`tests/*`	Tests unitaires avec AWS mocké	moto, fixtures, couverture
`.github/workflows/ci.yml`	Pipeline d'intégration continue	DevSecOps
`Dockerfile`	Packaging conteneurisé	Multi-stage, non-root, supply chain

1.6 Glossaire (à connaître par cœur pour un entretien)

- **Modèle de responsabilité partagée** — AWS sécurise l'infrastructure ; le client sécurise ses configurations, données et accès.
- **CIS AWS Foundations Benchmark** — référentiel de ~60 contrôles de sécurité AWS, standard de fait. Chaque check que tu écris doit citer son ID CIS.
- **ASFF** (*AWS Security Finding Format*) — le schéma JSON qu'attend AWS Security Hub. Produire de l'ASFF = ton outil s'intègre à l'écosystème AWS.
- **Finding** — un résultat d'évaluation : ressource + check + statut (PASS/FAIL) + sévérité + remédiation.
- **Sévérité** — CRITICAL / HIGH / MEDIUM / LOW / INFORMATIONAL (aligne-toi sur ASFF).
- **Moindre privilège** — ton scanner doit tourner avec la policy managée `SecurityAudit` (ou `ReadOnlyAccess`), jamais avec `AdministratorAccess`.
- **Confused deputy / AssumeRole + ExternalId** — comment un outil tiers accède à un compte client sans clé statique.
- **Drift** — écart entre l'état voulu et l'état réel de l'infrastructure.
- **Shift left** — détecter la misconfiguration au plus tôt (dans la CI, sur le code Terraform) plutôt qu'en production.

1.7 Règles de sécurité personnelles pour ce projet

1. **Jamais** de clé AWS en clair dans le code, un test, un commit, un screenshot. Utilise des profils AWS CLI ou des variables d'environnement.
2. Le labo Terraform crée des ressources **volontairement vulnérables** : compte AWS dédié, jamais de données réelles, `terraform destroy` systématique après usage.
3. Mets un **budget AWS** avec alerte à 5 $ dès le jour 1.
4. Ne scanne jamais un compte qui ne t'appartient pas sans autorisation écrite.
5. `.gitignore` couvre déjà `.tfstate`, `.env`, `credentials` — vérifie avant chaque `git push` avec `git diff --cached`.

2. Roadmap : les 10 phases

Chaque phase suit le même format :

- **Objectif** — ce que tu produis
- **Étapes** — le découpage concret
- **Ce que tu apprends** — la valeur ajoutée réelle pour ton profil
- **Terminé quand** — la définition de « fini » (DoD), pour ne pas t'éterniser

Ordre recommandé : 0 ? 1 ? 2 ? 3 ? 4 ? 5 ? 7 ? 6 ? 8 ? 9 ? 10. (La phase 7 « tests » arrive tôt volontairement : tester avec `moto` te permet de développer sans compte AWS. Le labo Terraform, phase 6, coûte de l'argent — tu le fais quand le code marche déjà.)

Estimation totale : **60 à 90 heures** de travail réel. C'est un projet de semestre, pas un week-end. Le viser en un week-end est la meilleure façon de l'abandonner.

Phase 0 — Fondations du projet

Objectif : un paquet Python installable, propre, avec les outils qualité.

Étapes

1. Remplir `pyproject.toml` : métadonnées, dépendances (`boto3`, `typer`, `rich`, `pydantic`), dépendances dev (`pytest`, `pytest-cov`, `moto`, `ruff`, `mypy`), build-system, layout `src/`, entry point console `cspm-lite` = `cspm_lite.cli:app`.
2. Créer un environnement virtuel et installer en mode éditable : `python -m venv .venv && . .venv/bin/activate && pip install -e ".[dev]"`.
3. Configurer `ruff` (lint + format) et `mypy` (mode strict) dans `pyproject.toml`.
4. Écrire un `README.md` minimal : titre, une phrase, badge CI, installation.
5. Vérifier que `import cspm_lite` fonctionne et que `cspm-lite --help` s'exécute.

Ce que tu apprends

Le packaging Python moderne (PEP 621, `pyproject.toml`) est une compétence que 80 % des étudiants n'ont pas : ils écrivent des scripts, pas des paquets. Le layout `src/` évite la classe de bugs « ça marche chez moi parce que le module est dans le dossier courant ». `ruff` et `mypy` t'apprennent à écrire du Python typé, ce qui est maintenant la norme en entreprise.

Terminé quand : `pip install -e ".[dev]"`, `ruff check .`, `mypy src` et `pytest` passent tous les quatre sans erreur.

Phase 1 — Le modèle de données et le moteur

Objectif : le cœur logique, indépendant d'AWS. C'est la phase la plus importante du projet — un mauvais modèle ici pollue tout le reste.

Étapes

1. `engine/models.py` :

- `Severity` — `Enum` : `CRITICAL`, `HIGH`, `MEDIUM`, `LOW`, `INFORMATIONAL`.
- `Status` — `Enum` : `PASS`, `FAIL`, `WARNING`, `NOT_AVAILABLE`.
- `Resource` — `id` (ARN), `type` (`AWS::S3::Bucket`), `region`, `account_id`, `name`, `tags`, `raw: dict` (la réponse brute de l'API).
- `Finding` — `check_id`, `title`, `description`, `severity`, `status`, `resource`, `remediation`, `references` (liens CIS/doc AWS), `detected_at`.

- `ScanResult` — `account_id`, `started_at`, `finished_at`, `findings`, plus des propriétés calculées : `counts_by_severity`, `pass_rate`, `score`.

2. `engine/registry.py` :

- un décorateur `@check(id=..., title=..., severity=..., service=..., cis=...)` qui enregistre la fonction dans un dictionnaire global ;
- une fonction `get_checks(services=None, severity_min=None)` pour filtrer ;
- une fonction `run_checks(inventory) -> list[Finding]` qui exécute et **capture les exceptions** (un check qui plante ne doit jamais casser le scan complet).

3. Un check bidon `dummy_always_pass` pour valider la mécanique, plus son test.

Ce que tu apprends

- **Modélisation de données** : concevoir un schéma stable qui survivra à 50 checks et 4 formats de sortie. C'est du design, pas du code.
- Le **pattern registry** (ou plugin) : le même mécanisme que les routes Flask (`@app.route`), les fixtures pytest, les tâches Celery. Comprendre le décorateur qui enregistre est un vrai palier en Python.
- **Isolation des pannes** : un moteur de règles robuste ne s'arrête pas au premier crash. C'est une réflexion d'ingénieur, pas d'étudiant.

Terminé quand : tu peux enregistrer un check avec un décorateur, l'exécuter via `run_checks()`, obtenir un `Finding`, et le sérialiser en JSON — sans qu'aucune ligne de `boto3` n'existe encore dans le projet.

Phase 2 — Authentification et collecteurs

Objectif : parler à l'API AWS proprement.

Étapes

1. `auth.py` :

- `get_session(profile=None, role_arn=None, region=None)` qui retourne une `boto3.Session` ;
- support de l'`AssumeRole` via STS (avec `ExternalId` optionnel) ;
- `whoami()` via `sts.get_caller_identity()` ? account_id, ARN de l'appelant ;
- `list_regions()` via EC2 pour le scan multi-région ;
- une config `botocore.Config` avec `retries={"mode": "adaptive", "max_attempts": 10}`.

2. `collectors/scan_s3.py` : `list_buckets`, puis pour chaque bucket `get_bucket_location`, `get_public_access_block`, `get_bucket_acl`, `get_bucket_policy`, `get_bucket_encryption`, `get_bucket_versioning`, `get_bucket_logging`. Gérer les `ClientError` (`NoSuchBucketPolicy`, `ServerSideEncryptionConfigurationNotFoundError` sont *normales*, pas des bugs).
3. `collectors/scan_iam.py` : users, access keys + `get_access_key_last_used`, MFA devices, policies attachées, `get_account_password_policy`, et surtout le **credential report** (`generate_credential_report` puis `get_credential_report`, avec attente active — l'API est asynchrone et retourne un CSV en base64).
4. `collectors/scan_ec2.py` : `describe_security_groups`, `describe_instances`, `describe_volumes`, `describe_vpcs`, `describe_flow_logs` — pour **toutes** les régions, en parallèle avec `concurrent.futures.ThreadPoolExecutor`.
5. `collectors/scan_cloudtrail.py` : `describe_trails`, `get_trail_status`, `get_event_selectors`.
6. Un `Collector` protocol/ABC commun : `name`, `service`, `collect(session) -> list[Resource]`.

Ce que tu apprends

- **L'API cloud dans la vraie vie** : la pagination (`get_paginator`), le throttling et le backoff exponentiel, les erreurs qui ne sont pas des erreurs, les endpoints globaux (IAM, S3 ListBuckets) contre régionaux.
- **La concurrence en Python** : le scan multi-région est *I/O bound*, donc les threads sont le bon outil (le GIL n'est pas un problème ici). Savoir expliquer pourquoi threads et pas `multiprocessing` est une question d'entretien classique.
- **IAM en profondeur** : la chaîne de credentials boto3, `AssumeRole`, les credentials temporaires STS, le

problème du *confused deputy* et pourquoi l'`ExternalId` existe.
- **Le credential report** : une API asynchrone qui rend un CSV base64. C'est inélégant et c'est exactement le genre de chose qu'on rencontre en entreprise.

Terminé quand : `cspm-lite scan --profile lab --dry-run` liste tes ressources réelles sur toutes les régions actives, sans crash, en moins de 60 secondes.

Phase 3 — Les checks de sécurité (le cœur du projet)

Objectif : 25 à 35 règles, chacune mappée sur un contrôle CIS.

Pour **chaque** check, écris systématiquement : l'ID CIS, le risque réel (« ce qui peut arriver »), la remédiation, et un test unitaire. Un check sans explication du risque n'a aucune valeur pédagogique.

s3_checks.py

ID	Check	Risque
S3-001	Block Public Access activé (4 flags)	Fuite de données publique
S3-002	ACL n'accorde rien à `AllUsers` / `AuthenticatedUsers`	Bucket lisible par le monde
S3-003	Bucket policy sans `Principal: "*"` sans condition	Idem, via policy
S3-004	Chiffrement par défaut activé (SSE-S3 ou SSE-KMS)	Données au repos en clair
S3-005	Versioning activé	Pas de récupération après ransomware/suppression
S3-006	Access logging activé	Aucune trace d'accès en cas d'incident
S3-007	Policy imposant TLS (`aws:SecureTransport: false` ? Deny)	Données en transit en clair
S3-008	MFA Delete activé (bonus)	Suppression d'objets sans second facteur

iam_checks.py

ID	Check	Risque	CIS
IAM-001	Le compte root n'a pas de clé d'accès	Compromission totale du compte	1.4
IAM-002	MFA activé sur le root	Idem	1.5
IAM-003	MFA activé sur tous les utilisateurs console	Vol de mot de passe = accès	1.10
IAM-004	Aucune clé d'accès de plus de 90 jours	Fenêtre d'exposition	1.14
IAM-005	Aucune credential inutilisée depuis 45 jours	Surface d'attaque dormante	1.12
IAM-006	Politique de mot de passe conforme (14 car., etc.)	Force brute	1.8
IAM-007	Aucun utilisateur avec `AdministratorAccess` attaché directement	Violation du moindre privilège	1.16
IAM-008	Aucune policy custom avec `Action: "*"` + `Resource: "*"`	Escalade de privilèges	1.16
IAM-009	Support role existant	Réponse à incident	1.17
IAM-010	Une seule clé active par utilisateur	Gestion des secrets	—

net_checks.py

ID	Check	Risque
NET-001	Aucun SG n'ouvre le port 22 sur `0.0.0.0/0`	SSH exposé, force brute
NET-002	Idem pour 3389 (RDP)	RDP exposé

NET-003	Idem pour 3306/5432/27017/6379 (bases)	Base de données exposée
NET-004	Aucun SG avec `-1` (tous ports) sur `0.0.0.0/0`	Ouverture totale
NET-005	Le security group `default` n'autorise aucun trafic	CIS 5.3
NET-006	VPC Flow Logs activés sur tous les VPC	Aucune visibilité réseau
NET-007	Volumes EBS chiffrés	Données au repos
NET-008	Pas d'IP publique sur les instances non prévues	Exposition
NET-009	IMDSv2 obligatoire sur les instances EC2	SSRF ? vol de credentials

`NET-009` mérite une attention particulière : c'est la faille de la brèche Capital One (2019, 100 millions de dossiers clients). Une SSRF permettait de lire les credentials IAM depuis `169.254.169.254`. IMDSv2 corrige ça. Sache raconter cette histoire en entretien.

log_checks.py

ID	Check	Risque	CIS
LOG-001	Au moins un trail CloudTrail multi-région activé	Aucun audit	3.1
LOG-002	Validation d'intégrité des fichiers de log activée	Falsification des logs	3.2
LOG-003	Le bucket des logs CloudTrail n'est pas public	Fuite des logs d'audit	3.3
LOG-004	Logs chiffrés avec KMS	Confidentialité	3.5
LOG-005	Intégration CloudWatch Logs	Pas d'alerte temps réel	3.4
LOG-006	Accès en écriture au bucket de logs journalisé	Traçabilité	3.6

Ce que tu apprends

C'est ici que se concentre la valeur cybersécurité. Tu n'apprends pas « du Python » : tu apprends **pourquoi** un bucket public a coûté des dizaines de millions à Capital One, Accenture, Verizon ; **pourquoi** une clé de 3 ans est le premier vecteur d'attaque cloud ; **comment** un attaquant enchaîne SSRF ? IMDS ? credentials ? `AssumeRole` ? exfiltration. Tu construis un vocabulaire d'entretien que très peu d'étudiants possèdent.

Tu apprends aussi à lire un **référentiel de conformité** et à le traduire en code — c'est littéralement le métier d'un ingénieur GRC/Cloud Security.

Terminé quand : au moins 25 checks implémentés, chacun avec son ID CIS, son texte de remédiation et son test unitaire.

Phase 4 — Les reporters

Objectif : transformer une liste de findings en quelque chose d'utilisable.

Étapes

1. `json_reporter.py` — sortie machine, schéma stable, versionné (`schema_version`).
2. `markdown_reporter.py` — tableau récapitulatif + détail par sévérité. C'est ce qu'on colle dans un commentaire de pull request.
3. `html_reporter.py` — un fichier HTML autonome (CSS et JS inline, aucune dépendance externe) : score global, répartition par sévérité, tableau filtrable, détail dépliable par finding.
4. `asff_reporter.py` — format AWS Security Hub : `SchemaVersion`, `Id`, `ProductArn`, `GeneratorId`, `Types`, `Severity.Label`, `Resources[]`, `Compliance.Status`, `Remediation.Recommendation`. Valide le résultat contre la doc AWS.
5. Un **scoring de posture** : par exemple `score = 100 * somme(poids des PASS) / somme(poids de tous)`, où le poids dépend de la sévérité (CRITICAL=10, HIGH=5, MEDIUM=3, LOW=1).

Ce que tu apprends

- La **séparation données / présentation** : un même `ScanResult`, quatre sorties.
- L'**interopérabilité** : produire de l'ASFF, c'est comprendre qu'un outil ne vit pas seul mais dans un écosystème. C'est la différence entre un script et un produit.
- La **dataviz** : choisir ce qu'on montre. Un rapport avec 300 findings non triés est inutile ; un rapport avec « 3 critiques, voici lesquelles » est actionnable. La règle : un rapport doit répondre à « qu'est-ce que je corrige en premier ? ».

Terminé quand : `cspm-lite scan --format html -o report.html` produit un fichier que tu serais fier de montrer à un recruteur, sans retouche.

Phase 5 — La CLI

Objectif : une interface que quelqu'un d'autre que toi peut utiliser.

Étapes

1. `cli.py` avec **Typer** :
- `scan` — options `--profile`, `--role-arn`, `--region` (répétable), `--service`, `--severity-min`, `--format`, `--output`, `--fail-on`.
- `list-checks` — tableau Rich de tous les checks (id, service, sévérité, CIS).
- `version`.
2. Une barre de progression **Rich** pendant la collecte.
3. Fichier de configuration `.cspm.yml` : checks désactivés, ressources exclues (allowlist avec justification et date d'expiration), sévérité minimale.
4. **Codes de sortie** : `0` = OK, `1` = findings au-dessus du seuil `--fail-on`, `2` = erreur d'exécution. C'est ce qui permet d'utiliser l'outil dans une CI.
5. Logging structuré avec `--verbose` / `--quiet`.

Ce que tu apprends

- L'**UX développeur** : la différence entre un outil qu'on utilise et un script qu'on abandonne tient souvent aux messages d'erreur et aux valeurs par défaut.
- Les **exit codes** : c'est le contrat entre ton outil et n'importe quel pipeline.
- La **gestion d'exceptions par allowlist** : toute exception de sécurité doit être documentée, justifiée et datée. C'est une notion de gouvernance, pas de code.

Terminé quand : `cspm-lite --help` est compréhensible sans lire le code, et `cspm-lite scan --fail-on HIGH` renvoie bien `1` quand il trouve un finding HIGH.

Phase 6 — Le labo Terraform

Objectif : une infrastructure AWS volontairement vulnérable, reproductible, détruite en une commande.

Attention : **cette phase crée de vraies ressources AWS et peut coûter de l'argent.** Fais-la après la phase 7 (tests avec moto), et pose un budget avec alerte avant.

Étapes

1. `main.tf` — provider AWS, `required_version`, `default_tags` (`Project = "cspm-lite-lab"`, `Owner`, `AutoDelete = "true"`).
2. `variables.tf` — région, préfixe de nommage, flag `enable_expensive_resources`.
3. `vulnerable_s3.tf` — un bucket sans Block Public Access, sans chiffrement, sans versioning, avec une policy publique en lecture.
4. `vulnerable_iam.tf` — un utilisateur avec `AdministratorAccess`, une policy custom `"Action": "*"` / `"Resource": "*"`, une clé d'accès, aucun MFA.
5. `vulnerable_sg.tf` — un security group avec 22, 3389 et 3306 ouverts sur `0.0.0.0/0`.

6. `outputs.tf` — les identifiants des ressources créées, pour les tests.
7. Un `README` de labo : `terraform init/plan/apply`, puis **toujours** `terraform destroy`.

Ce que tu apprends

- **Terraform** est probablement la compétence la plus demandée sur le marché cloud après le cloud lui-même. Tu apprends le cycle `plan/apply/destroy`, l'état, les variables, les outputs, le graphe de dépendances.
- Le **couple attaque/défense** : tu crées la vulnérabilité *et* le détecteur. Tu vérifies que ton check S3-001 lève bien un FAIL sur ton bucket. C'est la boucle de validation d'un vrai outil de sécurité.
- La **gestion des coûts cloud** (FinOps) : tags, budget, destruction systématique. Savoir dire « j'ai posé un budget et un tag AutoDelete » impressionne en entretien.

Terminé quand : `terraform apply` puis `cspm-lite scan` produit au moins 10 findings FAIL attendus, et `terraform destroy` ne laisse rien derrière.

Phase 7 — Tests et qualité

Objectif : développer sans compte AWS, et prouver que ça marche.

Étapes

1. `conftest.py` — fixtures : credentials AWS factices (variables d'environnement), décorateur `mock_aws` de **moto**, inventaires d'exemple.
2. `test_collectors.py` — avec moto : créer un bucket, lancer le collector, vérifier la `Resource` produite.
3. `test_checks.py` — tests **purs**, sans AWS : on injecte un inventaire fabriqué à la main et on vérifie le `Finding`. Un test PASS *et* un test FAIL par check.
4. `pytest.ini` / config dans `pyproject.toml` : couverture minimale à 80 % (`--cov=cspm_lite --cov-fail-under=80`).
5. Tests paramétrés (`@pytest.mark.parametrize`) pour les variantes de ports dans les checks réseau.

Ce que tu apprends

- **moto** : simuler AWS en local. C'est la technique qui permet de développer du code cloud sans facture ni risque, et c'est utilisé partout en entreprise.
- La **pyramide de tests** : beaucoup de tests unitaires rapides sur la logique pure (les checks), peu de tests d'intégration lents (les collectors). Ton architecture de la phase 1 rend ça possible — c'est le retour sur investissement du design.
- La **couverture** comme indicateur, pas comme objectif.

Terminé quand : `pytest --cov` passe à plus de 80 %, et tu peux développer un nouveau check de bout en bout sans jamais toucher à AWS.

Phase 8 — CI/CD, Docker et supply chain

Objectif : industrialiser.

Étapes

1. `.github/workflows/ci.yml` :

- déclencheurs `push` et `pull_request` ;
- job `lint` : `ruff check`, `ruff format --check`, `mypy src` ;
- job `test` : matrice Python 3.11 / 3.12, upload de la couverture ;
- job `build` : image Docker, scan **Trivy**, génération d'un **SBOM** (`anchore/sbom-action`), publication sur GHCR sur la branche `master` ;
- permissions minimales (`permissions: contents: read`), actions **épinglées par SHA** et non par tag.

2. `Dockerfile` multi-stage : un stage `builder` qui installe les dépendances, un stage final `python:3.12-slim` avec seulement le nécessaire, utilisateur non-root, `HEALTHCHECK`, `ENTRYPOINT ["cspm-lite"]`.

3. Un workflow bonus `security.yml` : `pip-audit` sur les dépendances + **Dependabot**.

Ce que tu apprends

- **DevSecOps concret** : la sécurité intégrée au pipeline plutôt qu'ajoutée après.
- La **sécurité de la supply chain** : épingler les actions par SHA empêche un attaquant ayant compromis un tag d'exécuter du code dans ta CI. Le SBOM répond à « quelles dépendances ai-je réellement ? » — la question posée après Log4Shell.
- Le **Dockerfile multi-stage et non-root** : deux réflexes que beaucoup de développeurs seniors n'ont toujours pas.

Terminé quand : le badge CI est vert sur le README, et `docker run ghcr.io/<toi>/cspm-lite --help` fonctionne.

Phase 9 — Le volet data

Objectif : passer d'un scan ponctuel à un suivi de posture dans le temps. C'est la phase qui différencie ton projet de tous les clones de Prowler.

Étapes

1. Persistance des scans : une base **SQLite** (ou des fichiers **Parquet** + **DuckDB**) avec un schéma `scans` / `findings`.
2. Une commande `cspm-lite history` : évolution du score, findings nouveaux, findings résolus, findings persistants entre deux scans (un `diff` de posture).
3. Des métriques de sécurité réelles :
- **MTTR** — temps moyen de remédiation par sévérité ;
- **taux de récidive** — findings qui réapparaissent après correction ;
- **backlog par service** — où se concentre la dette de sécurité.
4. Un notebook Jupyter ou un dashboard HTML de tendance (score dans le temps, top 10 des checks les plus en échec).
5. Bonus : export vers Prometheus/OpenMetrics pour un dashboard Grafana.

Ce que tu apprends

- Un vrai **mini-pipeline data** : ingestion ? normalisation ? stockage colonnaire ? agrégation ? visualisation. C'est de l'analytics engineering.
- La **modélisation temporelle** : distinguer l'état (snapshot) de l'événement (apparition/disparition d'un finding) est le cœur de la modélisation dimensionnelle.
- Les **métriques de sécurité** : savoir qu'on pilote la sécurité au MTTR et au taux de récidive, pas au nombre brut de vulnérabilités, est un raisonnement de manager.

Terminé quand : tu peux produire un graphique montrant l'évolution du score de posture sur au moins 5 scans successifs.

Phase 10 — Finition et valorisation

Objectif : que le projet serve ta carrière.

Étapes

1. `README.md` complet : problème résolu, capture d'écran du rapport HTML, GIF de démo (`asciinema` ou `vhs`), installation, usage, liste des checks, architecture, avertissement légal.
2. Une **documentation d'architecture** : 1 à 2 pages expliquant *pourquoi* collecte et évaluation sont séparées, quels compromis tu as faits (ADR — Architecture Decision Records).
3. Un **article** (blog, LinkedIn, Medium) : « J'ai construit un CSPM en Python pour comprendre la sécurité AWS de l'intérieur ». Raconte un check en détail, avec l'incident réel qu'il aurait évité.
4. Une **section CV** de 3 lignes (voir `05-competences-et-valorisation.md`).

5. Un **tag de release** `v1.0.0` et une page GitHub Releases.

Ce que tu apprends

Un projet non raconté n'existe pas. Savoir expliquer ton architecture en 3 minutes vaut autant que l'avoir codée. Écrire l'article te force à vérifier que tu as vraiment compris.

Terminé quand : un inconnu peut cloner, installer, lancer et comprendre le projet en moins de 10 minutes, sans te poser de question.

Récapitulatif

Phase	Sujet	Effort	Axe dominant
0	Fondations	3-5 h	Génie logiciel
1	Modèle + moteur	6-8 h	Génie logiciel / Data
2	Auth + collectors	10-14 h	Cloud
3	Checks de sécurité	15-20 h	**Cybersécurité**
4	Reporters	6-8 h	Data
5	CLI	4-6 h	Génie logiciel
6	Labo Terraform	5-7 h	Cloud / IaC
7	Tests	6-8 h	Génie logiciel
8	CI/CD + Docker	5-7 h	DevSecOps
9	Volet data	8-12 h	**Data**
10	Finition	4-6 h	Communication

3. Backlog : 30 issues prêtes à créer

gh (GitHub CLI) n'était pas installé sur cette machine, donc je n'ai pas pu lire ni créer les issues directement. Deux options :

1. **Automatique** — installegh, authentifie-toi, puis lance `bash learning/scripts/create_issues.sh`. ``bash sudo apt install gh # ou: https://cli.github.com gh auth login bash learning/scripts/create_issues.sh ``
2. **Manuel** — copie-colle les issues ci-dessous dans GitHub.

Labels à créer d'abord : `phase-0` ..`phase-10`, `cloud`, `security`, `data`, `devops`, `tests`, `docs`, `good-first-issue`.

Conseil : crée aussi un **GitHub Project** (vue tableau) avec les colonnes `Backlog` / `En cours` / `En revue` / `Terminé`. Un board actif visible sur ton profil est un signal fort pour un recruteur.

Phase 0 — Fondations

#1 —Remplir `pyproject.toml` et rendre le paquet installable `labels: phase-0, devops` Métadonnées PEP 621, dépendances runtime (`boto3`, `typer`, `rich`, `pydantic`) et dev (`pytest`, `pytest-cov`, `moto`, `ruff`, `mypy`), layout `src/`, entry point `cspm-lite` = `cspm_lite.cli:app`. *DoD* : `pip install -e ".[dev]"` fonctionne et `import cspm_lite` passe.

#2 —Configurer ruff et mypy en mode strict `labels: phase-0, devops, good-first-issue` Sections `[tool.ruff]` et `[tool.mypy]` dans `pyproject.toml`. `mypy` en `strict`. *DoD* : `ruff check . && mypy src` sortent en 0.

#3 —README initial `labels: phase-0, docs, good-first-issue` Titre, pitch d'une phrase, avertissement légal (usage sur ses propres comptes uniquement), installation, roadmap en 3 lignes.

Phase 1 — Modèle et moteur

#4 —`engine/models.py` : Severity, Status, Resource, Finding, ScanResult `labels: phase-1, data` Enums alignés sur ASFF. `Resource` conserve la réponse brute dans `raw`. `ScanResult` expose `counts_by_severity`, `pass_rate` et `score`. *DoD* : sérialisation JSON aller-retour testée.

#5 —`engine/registry.py` : décorateur `@check` et registre `labels: phase-1` `@check(id, title, severity, service, cis, remediation)` enregistre la fonction. `get_checks(services, severity_min)` filtre. `run_checks(inventory)` exécute. *DoD* : un check factice est découvert et exécuté par le registre.

#6 —Isolation des pannes dans le moteur `labels: phase-1` Un check qui lève une exception produit un `Finding` de statut `NOT_AVAILABLE` avec le message d'erreur, sans interrompre les autres checks. *DoD* : un test avec un check qui fait `raise` prouve que le scan continue.

#7 —Scoring de posture `labels: phase-1, data` Score pondéré par sévérité (CRITICAL=10, HIGH=5, MEDIUM=3, LOW=1). Documenter la formule dans le README.

Phase 2 — Auth et collecteurs

#8 —`auth.py` : sessions, profils, AssumeRole, STS `labels: phase-2, cloud, security` `get_session(profile, role_arn, external_id, region)`, `whoami()`, `list_regions()`, retries adaptatifs via `botocore.Config`. *DoD* : `cspm-lite whoami` affiche l'account id et l'ARN de l'appelant.

#9 —Interface commune `Collector` `labels: phase-2` Protocol avec `name`, `service`, `collect(session) -> list[Resource]`.

#10 —`collectors/scan_s3.py` `labels: phase-2, cloud` Buckets + location, public access block, ACL, policy, chiffrement, versioning, logging. Traiter `NoSuchBucketPolicy` et `ServerSideEncryptionConfigurationNotFoundError` comme des états normaux.

#11 —`collectors/scan_iam.py` avec credential report `labels: phase-2, cloud, security` Users, clés + `get_access_key_last_used`, MFA, policies attachées, password policy, et le credential report (API asynchrone ? CSV base64, avec attente et timeout).

#12 —`collectors/scan_ec2.py` multi-région et parallèle `labels: phase-2, cloud` SG, instances, volumes, VPC, flow logs sur toutes les régions actives via `ThreadPoolExecutor`. Justifier le choix threads vs processus dans le docstring. *DoD* : scan complet de 17 régions en moins de 60 s.

#13 —`collectors/scan_cloudtrail.py` `labels: phase-2, cloud` `describe_trails`, `get_trail_status`, `get_event_selectors`.

#14 —Gestion du throttling et des permissions manquantes `labels: phase-2, cloud` `AccessDenied` sur un appel ne doit pas tuer le scan : le collector remonte un avertissement et continue. Documenter la policy IAM minimale requise (`SecurityAudit`).

Phase 3 — Checks de sécurité

#15 —`s3_checks.py` : 8 checks `labels: phase-3, security` S3-001 à S3-008 (voir `02-roadmap.md`). Chaque check porte son ID CIS, son texte de remédiation et un lien de référence. *DoD* : 2 tests par check (un PASS, un FAIL).

#16 —`iam_checks.py` : 10 checks `labels: phase-3, security` IAM-001 à IAM-010. Le check « clés > 90 jours » doit s'appuyer sur le credential report, pas sur une heuristique.

#17 —`net_checks.py` : 9 checks `labels: phase-3, security` NET-001 à NET-009. Traiter les plages CIDR correctement avec le module `ipaddress` (`0.0.0.0/0` mais aussi `::/0` en IPv6, et les plages trop larges type `/8`).

#18 —`log_checks.py` : 6 checks `labels: phase-3, security` LOG-001 à LOG-006, mappés sur CIS section 3.

#19 —Catalogue des checks en Markdown généré `labels: phase-3, docs` Une commande qui génère `docs/CHECKS.md` depuis le registre : id, titre, sévérité, CIS, remédiation. La doc ne peut plus diverger du code.

Phase 4 — Reporters

#20 —`json_reporter.py` avec schéma versionné `labels: phase-4, data`

#21 —`markdown_reporter.py` pour les commentaires de PR `labels: phase-4, data` Résumé chiffré en tête, puis détail groupé par sévérité, repliable (`<details>`).

#22 —`html_reporter.py` : rapport autonome `labels: phase-4, data` Un seul fichier, CSS et JS inline, aucun CDN. Score, répartition par sévérité, tableau triable et filtrable. Doit rester lisible en mode sombre.

#23 —`asff_reporter.py` : intégration AWS Security Hub `labels: phase-4, cloud, security` Champs ASFF obligatoires, validation du format contre la doc AWS.

Phase 5 — CLI

#24 —CLI Typer : `scan`, `list-checks`, `version`, `whoami` `labels: phase-5` Options complètes, barre de progression Rich, sortie colorée.

#25 —Fichier de configuration `.cspm.yml` et exclusions `labels: phase-5, security` Exclusions avec champs obligatoires `reason` et `expires_at` ; une exclusion expirée redevient un finding actif.

#26 —Codes de sortie et `--fail-on` `labels: phase-5, devops` `0` OK, `1` seuil dépassé, `2` erreur interne.

Phase 6 — Labo Terraform

#27 —Labo Terraform vulnérable (S3, IAM, SG) `labels: phase-6, cloud` `main.tf`, `variables.tf`, `outputs.tf` + les 3 fichiers vulnérables. `default_tags` obligatoires, README avec avertissement de coût et rappel `terraform destroy`. *DoD* : `apply` puis scan produit au moins 10 FAIL attendus ; `destroy` est propre.

Phase 7 — Tests

#28 —Fixtures moto et tests de collectors `labels: phase-7, tests` `conftest.py` avec credentials factices et `mock_aws`. Couverture minimale 80 %.

Phase 8 — CI/CD

#29 —Pipeline GitHub Actions + Dockerfile durci `labels: phase-8, devops, security` Jobs lint / test / build. Actions épinglées par SHA, permissions minimales, scan Trivy, SBOM. Dockerfile multi-stage, utilisateur non-root.

Phase 9 — Data

#30 —Historisation SQLite et commande `history` `labels: phase-9, data` Schéma `scans` / `findings`, diff entre deux scans (nouveaux / résolus / persistants), courbe d'évolution du score, MTTR par sévérité.

Comment travailler ces issues

- **Une issue = une branche = une pull request.** Même seul sur le projet. Tu t'entraînes au workflow réel et ton historique Git devient lisible.
- Nomme les branches `feat/15-s3-checks`, `fix/12-region-timeout`.
- Écris des messages de commit conventionnels : `feat(checks): add S3 public access block check`.
- Dans chaque PR, écris 3 lignes : ce que ça fait, pourquoi, comment tu l'as testé. C'est un entraînement direct à la revue de code en entreprise.
- Ferme l'issue avec `Closes #15` dans la description de la PR.

4. Fiches de concepts clés

Ces fiches sont ce que tu dois pouvoir expliquer **sans notes**. Chacune correspond à une question posée en entretien de stage/alternance cloud ou cybersécurité.

4.1 Le modèle de responsabilité partagée

AWS est responsable de la sécurité **du** cloud : datacenters, hyperviseur, réseau physique, disponibilité des services managés. Toi tu es responsable de la sécurité **dans** le cloud : IAM, configuration des services, chiffrement, données, patchs de tes instances.

Corollaire : quasiment toutes les brèches cloud médiatisées sont des erreurs client, pas des failles AWS. C'est précisément le créneau d'un CSPM.

Question type : « Qui est responsable si un bucket S3 fuit ? » ? le client, sans ambiguïté. S3 est privé par défaut depuis toujours ; le rendre public est une action délibérée.

4.2 IAM : les 6 notions à maîtriser

1. **Principal** — qui agit (user, rôle, service).
2. **Policy** — un document JSON `Effect` / `Action` / `Resource` / `Condition`. Il en existe plusieurs types : identity-based (attachée à un principal), resource-based (attachée à la ressource, ex. bucket policy), SCP (au niveau de l'organisation), permission boundary.
3. **Évaluation** — un `Deny` explicite gagne toujours. Sans `Allow` explicite, c'est refusé. L'ordre : Deny explicite > SCP > boundary > Allow.
4. **Rôle vs utilisateur** — un rôle n'a pas de credentials permanents ; on l'*assume* via STS et on reçoit des credentials temporaires. **C'est toujours la bonne réponse.** Un utilisateur IAM avec clé statique est un anti-pattern.
5. **AssumeRole + ExternalId** — quand un outil tiers (comme un CSPM SaaS) accède à ton compte, tu crées un rôle qu'il assume. L'`ExternalId` empêche le *confused deputy* : sans lui, un autre client du même prestataire pourrait deviner ton ARN et se faire accéder à ton compte par le prestataire.
6. **Moindre privilège** — ton scanner doit tourner avec `SecurityAudit`, une policy managée AWS en lecture seule orientée audit. Si tu demandes `AdministratorAccess` pour un outil de lecture, tu es toi-même le risque.

Chaîne de credentials boto3 (ordre de résolution) : paramètres explicites ? variables d'environnement ? fichier `~/.aws/credentials` ? rôle du conteneur (ECS) ? rôle de l'instance (IMDS). Savoir réciter cet ordre est une question d'entretien fréquente.

4.3 S3 : pourquoi les buckets fuient

Quatre mécanismes rendent un bucket public, et il faut les vérifier tous :

1. **ACL** — l'ancien mécanisme `AllUsers` = tout Internet. `AuthenticatedUsers` = **n'importe quel compte AWS au monde**, pas « mes utilisateurs » — c'est le piège classique.
2. **Bucket policy** — `"Principal": "*"` sans `Condition`.
3. **Block Public Access** — 4 flags (`BlockPublicAcls`, `IgnorePublicAcls`, `BlockPublicPolicy`, `RestrictPublicBuckets`). Activés, ils neutralisent 1 et 2. À activer au niveau du compte, pas seulement du bucket.
4. **Website hosting / presigned URLs** — moins fréquent mais réel.

Autres contrôles importants : chiffrement au repos (SSE-S3 gratuit, SSE-KMS pour la traçabilité des accès à la clé), versioning (récupération après suppression ou ransomware), access logging, et une policy qui refuse `aws:SecureTransport: false` pour imposer TLS.

Incidents réels à citer : Capital One (2019), Accenture (2017), Verizon (2017), Booz Allen Hamilton (2017) — tous des buckets mal configurés.

4.4 Réseau : security groups et NACL

- **Security Group** — pare-feu **stateful** au niveau de l'interface réseau. Autorise uniquement (pas de règle deny). Le trafic retour est automatiquement permis.
- **NACL** — pare-feu **stateless** au niveau du sous-réseau. Autorise et refuse. Il faut ouvrir explicitement le retour (ports éphémères).

Les erreurs qui comptent :

- `0.0.0.0/0` sur 22 (SSH) ou 3389 (RDP) — force brute garantie en quelques minutes ; la bonne réponse moderne est **AWS Systems Manager Session Manager**, qui supprime le besoin d'ouvrir SSH.
- `0.0.0.0/0` sur 3306, 5432, 27017, 6379 — base de données exposée à Internet.
- Le SG `default` laissé permissif : il s'applique par défaut à toute nouvelle ressource dont on oublie de spécifier le SG.
- Absence de **VPC Flow Logs** : après un incident, tu n'as aucune trace réseau.

4.5 IMDS et la brèche Capital One

Une instance EC2 peut lire ses propres credentials IAM temporaires sur `http://169.254.169.254/latest/meta-data/iam/security-credentials/`.

En 2019, Capital One : une application web derrière un WAF mal configuré était vulnérable à une **SSRF**. L'attaquante a fait interroger cette adresse par le serveur, récupéré les credentials du rôle de l'instance — un rôle sur-privilégié — et exfiltré 100 millions de dossiers clients depuis S3. Amende : 80 millions de dollars.

IMDSv2 corrige la classe de faille : il exige un token obtenu par une requête `PUT` avec un en-tête spécifique, ce qu'une SSRF simple ne peut pas produire. Rendre IMDSv2 obligatoire (`HttpTokens: required`) est le check NET-009.

Cette histoire contient trois leçons : SSRF, moindre privilège du rôle d'instance, et défense en profondeur. Sache la raconter en 90 secondes.

4.6 CloudTrail : sans logs, pas d'investigation

CloudTrail enregistre chaque appel d'API du compte. Sans lui, une intrusion est invisible et non-investigable.

Les points de contrôle :

- **Multi-région** — sinon un attaquant opère depuis une région non surveillée.
- **Validation d'intégrité des fichiers** — signature qui détecte l'altération des logs *a posteriori*. Un attaquant qui efface ses traces est détecté.
- **Bucket de logs non public et journalisé**.
- **Chiffrement KMS**.
- **Intégration CloudWatch Logs** — pour alerter en temps réel, pas juste archiver.

Différence à connaître : **CloudTrail** = plan de contrôle (qui a fait quoi sur l'API). **VPC Flow Logs** = plan réseau. **S3 access logs / CloudTrail data events** = plan données (qui a lu quel objet). Les trois sont complémentaires.

4.7 Le CIS AWS Foundations Benchmark

Un document public du Center for Internet Security, ~60 contrôles numérotés par section :

- **1.x** — IAM
- **2.x** — Stockage (S3, EBS, RDS)
- **3.x** — Logging (CloudTrail, Config)
- **4.x** — Monitoring (alarmes CloudWatch)
- **5.x** — Réseau (VPC, security groups)

Chaque contrôle a un niveau (Level 1 = essentiel, Level 2 = renforcé), un rationnel, une procédure d'audit et une procédure de remédiation.

Pourquoi c'est important pour toi : quand tu écris `cis="1.14"` sur ton check, tu ne fais pas de la décoration. Tu transformes ton outil en outil de **conformité**, et tu apprends le vocabulaire GRC (gouvernance, risque, conformité) que les entreprises utilisent. Autres référentiels à connaître de nom : NIST CSF, ISO 27001, PCI-DSS, SOC 2, et pour ton profil au Maroc/en France : la directive NIS2 et le RGPD.

4.8 ASFF : parler la langue de l'écosystème

Le *AWS Security Finding Format* est le schéma JSON de Security Hub. Champs essentiels : `SchemaVersion`, `Id`, `ProductArn`, `GeneratorId`, `AwsAccountId`, `Types`, `CreatedAt`, `Severity.Label`, `Title`, `Description`, `Resources[]`, `Compliance.Status`, `Remediation.Recommendation.Text`.

L'intérêt pédagogique : comprendre que la valeur d'un outil de sécurité ne vient pas seulement de ce qu'il détecte, mais de sa capacité à **s'insérer dans un flux existant** (SIEM, ticketing, Security Hub). Un outil qui produit un format propriétaire finit ignoré.

4.9 boto3 : les pièges de l'API AWS

- **Pagination** — `list_*` et `describe_*` renvoient souvent 50 ou 100 éléments maximum. Utilise toujours `client.get_paginator("list_users").paginate()`, jamais l'appel brut. C'est le bug numéro 1 des scripts AWS amateurs.
- **Throttling** — l'API renvoie `Throttling` / `RequestLimitExceeded` en cas de rafale. Configure `Config(retries={"mode": "adaptive", "max_attempts": 10})`.
- **Erreurs qui n'en sont pas** — `NoSuchBucketPolicy`, `ServerSideEncryptionConfigurationNotFoundError`, `NoSuchPublicAccessBlockConfiguration` signifient simplement « pas configuré ». Les traiter comme des exceptions fatales rend l'outil inutilisable.
- **Global vs régional** — IAM et `s3:ListBuckets` sont globaux ; EC2, CloudTrail, et les opérations par bucket sont régionaux. Un bucket doit être interrogé dans sa région d'origine.
- **Client vs Resource** — l'API `resource` est plus jolie mais incomplète et en maintenance réduite. Utilise `client` pour un outil sérieux.

4.10 moto : simuler AWS en local

`moto` intercepte les appels boto3 et simule les services AWS en mémoire.

```
from moto import mock_aws

@mock_aws
def test_public_bucket_is_flagged():
    s3 = boto3.client("s3", region_name="us-east-1")
```

```
    s3.create_bucket(Bucket="leaky")
    ...
```

Prérequis absolu : mettre de **fausses credentials** dans l'environnement du test (`AWS_ACCESS_KEY_ID=testing`, etc.) via une fixture `autouse`. Sinon, si moto échoue à intercepter, ton test taperait sur ton vrai compte AWS. C'est un piège classique et dangereux.

Limite : moto ne simule pas tout (le credential report IAM, par exemple, est partiel). Pour ces cas, teste la logique de parsing séparément avec un CSV d'exemple en fixture — ce qui prouve encore une fois l'intérêt d'avoir séparé collecte et évaluation.

4.11 Terraform : le vocabulaire minimum

- **Provider** — le plugin qui parle à AWS.
- **Resource** — une ressource déclarée.
- **State** (`terraform.tfstate`) — la correspondance entre le code et le monde réel. **Contient des secrets en clair** — d'où sa présence dans `.gitignore`. En équipe, on le met dans un backend S3 chiffré avec verrouillage DynamoDB.
- **Plan / Apply / Destroy** — le cycle. `plan` est un *dry-run*, à lire toujours.
- **Drift** — quelqu'un a modifié la ressource à la main ; `plan` le révèle.
- **Modules** — la réutilisation.

Concept lié pour ton axe sécurité : **Checkov / tfsec / Terrascan** analysent le code Terraform *avant* déploiement. C'est le « shift left » : le CSPM détecte en production, l'analyse IaC détecte au commit. Comprendre la complémentarité des deux est un très bon point d'entretien — et une extension naturelle de ton projet.

4.12 Docker et supply chain

- **Multi-stage** — le stage de build contient les compilateurs et outils ; l'image finale n'a que le runtime. Moins de surface d'attaque, image 5 à 10 fois plus petite.
- **Non-root** — `USER appuser`. Si un attaquant s'échappe de ton processus, il n'est pas root dans le conteneur.
- **Épingler les versions** — `python:3.12-slim` plutôt que `python:latest`, et à terme le digest `sha256:`.
- **Scanner l'image** — Trivy détecte les CVE des paquets système et Python.
- **SBOM** — l'inventaire des composants. Après Log4Shell (décembre 2021), la question « ai-je log4j quelque part ? » a pris des semaines à répondre dans la plupart des entreprises. Le SBOM répond en secondes.
- **Actions GitHub épinglées par SHA** — un tag Git est mutable ; un attaquant qui compromet un dépôt d'action peut re-pointer `v3` vers du code malveillant qui s'exécutera dans ta CI avec tes secrets. Cette attaque s'est produite plusieurs fois en conditions réelles.

5. Compétences acquises et valorisation

5.1 Ce que ce projet met réellement sur ton CV

Compétence	Où tu l'acquiers	Comment un recruteur la lit
Sécurité cloud AWS	Phase 3	« Connaît le CIS Benchmark, pas juste le nom d'AWS »
IAM avancé (STS, AssumeRole, policies)	Phases 2-3	Compétence rare chez un junior
boto3 / API AWS en production	Phase 2	Pagination, throttling, multi-région
Terraform / IaC	Phase 6	Compétence la plus demandée du marché cloud
Python d'ingénierie (typé, testé, packagé)	Phases 0-1-7	Distingue « sait coder » de « sait livrer »
Architecture logicielle (registry, séparation des couches)	Phase 1	Sujet de discussion technique en entretien
Tests avec mocks cloud (moto)	Phase 7	Très peu d'étudiants savent le faire
CI/CD et DevSecOps	Phase 8	Trivy, SBOM, actions épinglées
Sécurité de la supply chain	Phase 8	Sujet chaud post-Log4Shell / xz-utils
Pipeline data et métriques	Phase 9	Ton différenciateur data
Conformité et GRC	Phase 3	Vocabulaire des équipes sécurité

5.2 Formulations pour ton CV

À adapter, jamais copier tel quel — e**ne mets rien que tu ne saches défendre**.

> **cspm-lite —Scanner de posture de sécurité cloud AWS** (Python, boto3, Terraform, Docker) > Outil d'audit automatisé implémentant 30+ contrôles du CIS AWS Foundations > Benchmark (IAM, S3, réseau, logging). Architecture en couches collecte/évaluation > avec registre de règles extensible ; scan multi-région parallélisé ; rapports > JSON/HTML/Markdown et export ASFF vers AWS Security Hub. Labo Terraform > volontairement vulnérable pour la validation ; 85 % de couverture de tests avec > moto ; CI GitHub Actions avec lint, typage, scan de conteneur Trivy et SBOM.

Version courte pour LinkedIn :

> J'ai construit cspm-lite, un CSPM open source pour AWS : 30+ checks CIS, > collecte multi-région parallélisée, export Security Hub, labo Terraform vulnérable > et CI DevSecOps complète. Le but n'était pas de refaire Prowler mais de comprendre > de l'intérieur pourquoi les comptes cloud se font compromettre.

5.3 Les questions d'entretien que ce projet te prépare à gérer

Entraîne-toi à répondre en 2 minutes maximum à chacune :

Cloud / sécurité

1. Qu'est-ce que le modèle de responsabilité partagée ?
2. Comment un bucket S3 devient-il public ? Cite les quatre mécanismes.
3. Différence entre un rôle et un utilisateur IAM ? Lequel préfères-tu et pourquoi ?
4. Comment un outil tiers accède-t-il à un compte AWS client en sécurité ? (AssumeRole + ExternalId, et pourquoi l'ExternalId existe)
5. Raconte-moi une brèche cloud réelle et sa cause technique. (Capital One)

6. Comment détectes-tu qu'un attaquant a effacé ses traces ? (validation d'intégrité CloudTrail)
7. Security group vs NACL ?
8. Qu'est-ce qu'IMDSv2 résout ?
9. Comment donnerais-tu à ton scanner le droit de lire un compte sans le compromettre ?

Génie logiciel

10. Explique l'architecture de ton projet et pourquoi tu as séparé collecte et évaluation.
11. Comment testes-tu du code qui appelle AWS, sans compte AWS ?
12. Ton scan est lent : comment l'optimises-tu ? (threads car I/O bound, pagination, filtrage des régions actives)
13. Un check plante en production. Que se passe-t-il ? (isolation des pannes)

DevOps

14. Pourquoi un Dockerfile multi-stage ?
15. Pourquoi épingler les actions GitHub par SHA ?
16. Qu'est-ce qu'un SBOM et à quoi ça sert ?

Data

17. Comment mesures-tu si la posture de sécurité s'améliore ? (score dans le temps, MTTR, taux de récidive — pas le nombre brut de findings)
18. Comment modélises-tu l'historique des findings ? (snapshot d'état vs événements)

5.4 Certifications cohérentes avec ce projet

Le projet te prépare directement à plusieurs certifications. Ordre recommandé :

1. **AWS Certified Cloud Practitioner** (facultatif si tu es déjà à l'aise) — 100 $.
2. **AWS Certified Solutions Architect – Associate** — la plus reconnue, 150 $. Le projet couvre déjà IAM, S3, VPC, CloudTrail.
3. **AWS Certified Security – Specialty** — 300 $. Ton projet en couvre facilement 30 à 40 % du programme. C'est la certification la plus alignée.
4. **HashiCorp Terraform Associate** — 70 $, très accessible après la phase 6.
5. Côté sécurité générale : **CompTIA Security+** si tu vises un premier poste généraliste, ou le parcours **TryHackMe / HackTheBox** pour l'offensif.

Note importante : les certifications sans projet sont creuses, et le projet sans certification passe parfois les filtres RH difficilement. La combinaison des deux est ce qui fonctionne.

5.5 Extensions possibles (après la v1)

Quand le projet est fini, ces extensions le font passer d'un bon projet étudiant à un portfolio remarquable :

- **Multi-cloud** — ajouter un collecteur Azure ou GCP. Prouve que ton architecture était bonne : si ajouter Azure ne demande pas de réécrire le moteur, ton design tenait.
- **Analyse IaC** — scanner les fichiers Terraform avant déploiement (le « shift left »), en réutilisant tes checks.
- **Remédiation automatique** — un mode- `-fix` qui corrige les findings à faible risque (activer Block Public Access, par exemple), avec confirmation obligatoire. Attention : c'est là que se cachent les incidents ; exige un `--yes` explicite et un mode `--dry-run` par défaut.
- **Attack path analysis** — le sujet le plus intéressant intellectuellement : construire un graphe des relations IAM (qui peut assumer quel rôle, qui peut atteindre quelle donnée) et chercher les chemins d'escalade de privilèges. C'est ce que font Wiz et Cartography. Un graphe `networkx` suffit pour une v1.
- **Intégration Slack / ticketing** — un finding CRITICAL crée une issue GitHub ou un message Slack.
- **Mode serveur** — une API FastAPI + un scan planifié, pour un suivi continu.

5.6 Le plan de travail réaliste

Si tu es étudiant à temps plein, vise **6 à 8 heures par semaine sur 10 à 12 semaines**.

Semaines	Objectif	Livrable visible
1	Phases 0 et 1	Paquet installable, moteur fonctionnel
2-3	Phase 2	`cspm-lite scan` liste tes ressources réelles
4-6	Phase 3	25+ checks, le cœur du projet
7	Phases 4 et 5	Rapport HTML présentable, CLI complète
8	Phase 7	Tests à 80 %, badge de couverture
9	Phase 6	Labo Terraform validant les checks
10	Phase 8	CI verte, image Docker publiée
11-12	Phases 9 et 10	Historique, article, release v1.0.0

Règle anti-abandon : un commit par session de travail, même minuscule. Le graphe de contributions GitHub qui se remplit est un moteur psychologique réel, et un recruteur regarde la régularité autant que le contenu.

Règle anti-perfectionnisme : ne repars pas en arrière pour « améliorer » une phase terminée avant d'avoir atteint la phase 10. Un projet fini à 80 % vaut infiniment plus que trois phases parfaites et un projet mort.

6. Journal de bord et suivi

6.1 Pourquoi tenir un journal

Trois raisons concrètes, aucune n'est de la discipline pour la discipline :

1. **Tu retiens ce que tu écris.** Reformuler un concept le jour même le fixe durablement ; le relire passivement ne le fixe pas.
2. **Tu produis la matière de ton entretien.** Six mois plus tard, tu ne te souviendras pas pourquoi tu as choisi les threads plutôt qu'`asyncio`. Ton journal si.
3. **Tu produis la matière de ton article final** (phase 10) sans effort supplémentaire.

Règle : 5 à 10 lignes par session, pas plus. Un journal qui devient une corvée est un journal abandonné.

6.2 Modèle d'entrée

Copie ce bloc à chaque session dans `learning/journal/AAAA-MM-JJ.md` :

```
## 2026-08-03 — Phase 2, issue #10 (collector S3)

**Fait** :
-

**Bloqué sur / résolu comment** :
-

**Appris (le truc que je ne savais pas ce matin)** :
-

**Question ouverte** :
-

**Prochaine session** :
-
```

La ligne qui compte le plus est « **Appris** ». Si tu ne peux rien y écrire, la session était de l'exécution mécanique — ce n'est pas grave de temps en temps, mais si ça se répète trois fois, change de tâche.

6.3 Tableau de progression

Coche au fur et à mesure.

Phase 0 — Fondations

- [ ] `pyproject.toml` complet, `pip install -e ".[dev]"` fonctionne
- [ ] `ruff` et `mypy` configurés et verts
- [ ] README initial
- [ ] Premier commit sur une branche, première PR

Phase 1 — Modèle et moteur

- [ ] `models.py` : Severity, Status, Resource, Finding, ScanResult
- [ ] `registry.py` : décorateur `@check` + `get_checks` + `run_checks`
- [ ] Isolation des pannes (un check qui plante n'arrête pas le scan)
- [ ] Scoring de posture
- [ ] Sérialisation JSON testée

Phase 2 — Auth et collecteurs

- [ ] `auth.py` : session, profil, AssumeRole, `whoami`
- [ ] Retries adaptatifs configurés
- [ ] Interface `Collector`
- [ ] `scan_s3.py`
- [ ] `scan_iam.py` avec credential report
- [ ] `scan_ec2.py` multi-région parallélisé
- [ ] `scan_cloudtrail.py`
- [ ] `AccessDenied` géré proprement

Phase 3 — Checks

- [ ] 8 checks S3
- [ ] 10 checks IAM
- [ ] 9 checks réseau
- [ ] 6 checks logging
- [ ] Tous mappés CIS, avec remédiation
- [ ] `docs/CHECKS.md` généré automatiquement

Phase 4 — Reporters

- [ ] JSON versionné
- [ ] Markdown pour PR
- [ ] HTML autonome et présentable
- [ ] ASFF valide

Phase 5 — CLI

- [ ] `scan`, `list-checks`, `whoami`, `version`
- [ ] Barre de progression Rich
- [ ] `.cspm.yml` avec exclusions datées
- [ ] Exit codes et `--fail-on`

Phase 6 — Labo Terraform

- [ ] Budget AWS avec alerte posé **avant** tout `apply`
- [ ] `main.tf`, `variables.tf`, `outputs.tf`
- [ ] S3, IAM et SG vulnérables
- [ ] Scan du labo produit les FAIL attendus
- [ ] `terraform destroy` vérifié, aucune ressource orpheline

Phase 7 — Tests

- [ ] `conftest.py` avec credentials factices en `autouse`
- [ ] Tests de collectors avec moto
- [ ] Un test PASS et un test FAIL par check
- [ ] Couverture > 80 %

Phase 8 — CI/CD

- [ ] Workflow lint / test / build
- [ ] Actions épinglées par SHA, permissions minimales
- [ ] Dockerfile multi-stage non-root
- [ ] Scan Trivy et SBOM
- [ ] Image publiée sur GHCR

Phase 9 — Data

- [ ] Persistance SQLite ou Parquet

- [ ] Commande `history` avec diff entre scans
- [ ] Courbe d'évolution du score
- [ ] MTTR et taux de récidive

Phase 10 — Finition

- [ ] README complet avec capture et GIF
- [ ] Doc d'architecture / ADR
- [ ] Article publié
- [ ] Section CV rédigée
- [ ] Release `v1.0.0` taguée

6.4 Auto-évaluation : sais-tu expliquer… ?

À revoir tous les 15 jours. Coche seulement si tu peux répondre **à l'oral, sans notes, en moins de 2 minutes**.

- [ ] Le modèle de responsabilité partagée
- [ ] Les 4 façons dont un bucket S3 devient public
- [ ] Rôle vs utilisateur IAM, et pourquoi le rôle gagne
- [ ] AssumeRole + ExternalId et le problème du confused deputy
- [ ] L'ordre d'évaluation des policies IAM
- [ ] La chaîne de credentials boto3
- [ ] La brèche Capital One de bout en bout
- [ ] Ce que résout IMDSv2
- [ ] Security group vs NACL
- [ ] Pourquoi la validation d'intégrité CloudTrail existe
- [ ] Ce qu'est le CIS Benchmark et à quoi servent ses sections
- [ ] Ce qu'est l'ASFF et pourquoi tu le produis
- [ ] Pourquoi tu as séparé collecte et évaluation dans ton code
- [ ] Pourquoi des threads et pas des processus pour ton scan
- [ ] Comment tu testes du code AWS sans compte AWS
- [ ] Pourquoi un Dockerfile multi-stage et non-root
- [ ] Pourquoi épingler les actions GitHub par SHA
- [ ] Ce qu'est un SBOM
- [ ] Comment on mesure l'amélioration d'une posture de sécurité

6.5 Ressources de référence

Documentation officielle

- CIS AWS Foundations Benchmark — cisecurity.org (téléchargement gratuit après inscription)
- AWS Security Best Practices / Well-Architected Framework, pilier Sécurité
- Documentation boto3 — boto3.amazonaws.com
- ASFF — docs.aws.amazon.com/securityhub, section « finding format »

Code à lire (lire du code d'outil réel vaut dix tutoriels)

- **Prowler** — github.com/prowler-cloud/prowler : regarde `providers/aws/services/`, c'est exactement l'architecture que tu construis
- **Scout Suite** — github.com/nccgroup/ScoutSuite : les rulesets en JSON
- **Cartography** — github.com/lyft/cartography : le graphe d'infrastructure, pour l'extension attack-path
- **Checkov** — github.com/bridgecrewio/checkov : l'analyse IaC

Pratique offensive/défensive

- **flaws.cloud** et **flaws2.cloud** — challenges gratuits de sécurité AWS, à faire absolument avant la phase 3, ils donnent l'intuition de l'attaquant
- **CloudGoat** (Rhino Security Labs) — scénarios d'attaque AWS déployés en Terraform
- **AWS Skill Builder** — parcours gratuits, dont un labo sécurité

Veille

- tldr.sec (newsletter hebdomadaire, la meilleure du domaine)
- Le blog sécurité AWS
- Les rapports d'incidents de Datadog Security Labs et Wiz Research

7. Maximiser le bagage

Construire cspm-lite et **capitaliser** dessus sont deux choses différentes. Beaucoup d'étudiants finissent un projet solide et en ressortent avec trois lignes de CV qu'ils ne savent pas défendre. Cette partie traite uniquement de la conversion : comment transformer 80 heures de travail en compétence durable, démontrable et transférable.

7.1 Les quatre leviers

Le bagage réel ne vient pas du volume de code écrit. Il vient de quatre choses :

Profondeur. Un check S3 écrit en 10 minutes t'apprend une syntaxe boto3. Le même check, avec 30 minutes de plus pour comprendre les quatre mécanismes de publication d'un bucket et écrire la commande de remédiation, t'apprend la sécurité S3 pour dix ans. Le rendement de la profondeur est très supérieur à celui de la quantité.

Contraste attaque/défense. Tu n'écris un bon détecteur que si tu sais ce que l'attaquant en fait. Chaque contrôle que tu implémentes doit être accompagné d'un scénario d'exploitation que tu as compris — idéalement exécuté toi-même sur ton labo.

Restitution. Ce que tu n'as jamais expliqué à voix haute, tu ne le possèdes pas. La restitution (journal, article, présentation, revue de code) est ce qui transforme une information consultée en connaissance mobilisable sous pression, c'est-à-dire en entretien.

Preuve. Une compétence non prouvable ne se vend pas. Chaque compétence de ton CV doit être adossée à un lien (commit, PR, rapport, article) et à une anecdote de trois phrases. C'est l'objet du carnet de preuves (section 7.7).

7.2 L'extra mile, phase par phase

Pour chaque phase, voici le petit surinvestissement dont le rendement est disproportionné. Ce sont environ 25 heures supplémentaires au total, réparties sur tout le projet, qui changent la nature de ce que tu en retires.

Phase	Investissement supplémentaire	Ce que ça débloque
0	+2 h — écrire un **ADR** (Architecture Decision Record) justifiant tes choix : layout `src/`, pydantic ou dataclasses, ruff plutôt que flake8+black	Tu sais défendre chaque choix technique au lieu de dire « c'est ce que le tutoriel faisait »
1	+2 h — ajouter des **tests de propriété** avec `hypothesis` sur la sérialisation du modèle	Le property-based testing est rare chez un junior et impressionne en entretien technique
2	+3 h — construire la **policy IAM minimale exacte** de ton scanner, empiriquement : tourne avec une policy vide, note chaque `AccessDenied`, ajoute l'action, recommence	Tu pratiques le moindre privilège pour de vrai. C'est exactement la méthode des équipes sécurité en entreprise
3	+30 min par check — pour chaque contrôle, écrire la **commande AWS CLI de vérification manuelle** et la **commande de remédiation**	Tu produis un runbook opérationnel. Tu passes de « je détecte » à « je sais corriger », ce qui est le vrai métier
4	+2 h — pousser réellement tes findings dans un **AWS Security Hub** (le premier mois est gratuit) et les voir apparaître	Boucle fermée avec un service AWS réel. Très peu d'étudiants sont allés jusque-là
5	+2 h — faire tester ta CLI par un camarade **sans aucune explication**, noter chaque friction, corriger	Tu apprends l'UX développeur et la valeur du retour utilisateur brut
6	+3 h — écrire et exécuter un **scénario d'attaque complet** sur ton labo (clé exposée ? `AssumeRole` ? exfiltration S3), puis vérifier que ton scanner l'aurait empêché	C'est ton anecdote d'entretien la plus forte. Tu as attaqué et défendu la même infrastructure
7	+2 h — **mutation testing** avec `mutmut` : il modifie ton code et vérifie que tes tests échouent	Tu découvres que 80 % de couverture ne veut pas dire 80 % de tests utiles. Concept de niveau senior

8	+3 h — **signer tes images** avec `cosign` et générer une **provenance SLSA**	Sécurité de la supply chain au niveau de l'état de l'art 2024-2026, sujet très demandé
9	+3 h — définir des **SLO de sécurité** (« 100 % des findings CRITICAL corrigés sous 7 jours ») et les mesurer	Tu passes du technique au pilotage. C'est le langage des responsables sécurité
10	+4 h — enregistrer une **présentation de 10 minutes** de ton projet, la réécouter, la refaire	La compétence la plus sous-estimée. Tu répètes ton entretien avant de le passer

7.3 Les trois pistes parallèles

À mener **pendant** le projet, pas après. Elles coûtent 2 à 3 heures par semaine et doublent le rendement du temps passé à coder.

Piste 1 — Offensive (2 h toutes les deux semaines)

L'ordre compte :

1. **flaws.cloud** — 6 niveaux, gratuit, aucune installation. À faire **avant** la phase 3. Tu y découvres un bucket public, un rôle sur-privilégié, l'IMDS. Tes checks auront un sens après.
2. **flaws2.cloud** — deux pistes, attaquant et défenseur. Fais les deux.
3. **CloudGoat** (Rhino Security Labs) — scénarios déployés en Terraform dans ton propre compte. Fais au minimum `iam_privesc_by_rollback` et `ec2_ssrf`. C'est là que tu comprends physiquement l'escalade de privilèges IAM.
4. **Pacu** — le framework d'exploitation AWS. Lance-le contre ton propre labo Terraform. Regarder un outil offensif énumérer tes ressources change ta façon d'écrire des détections.

Après chaque scénario, une entrée de journal en trois lignes : ce que l'attaquant a fait, quel check de cspm-lite l'aurait bloqué, et si ce check n'existe pas, crée l'issue.

Piste 2 — Lecture de code réel (1 h par semaine)

Lire du code d'outil professionnel vaut dix tutoriels. Programme :

- **Semaines 1-4** — Prowler, dossier `prowler/providers/aws/services/s3/`. Compare leur structure de check à la tienne. Note trois différences et demande-toi pourquoi.
- **Semaines 5-6** — Prowler, le moteur : `prowler/lib/check/check.py`. Comment ils chargent les checks dynamiquement, comment ils gèrent les erreurs.
- **Semaines 7-8** — Scout Suite, les rulesets JSON. Une approche déclarative plutôt qu'impérative : quels compromis ?
- **Semaines 9-10** — Cartography (Lyft) : le modèle de graphe Neo4j de l'infrastructure. Prépare l'extension attack-path.
- **Semaines 11-12** — Checkov : l'analyse statique de Terraform.

Note dans ton journal chaque idée volée et chaque idée rejetée avec sa raison. Ce carnet devient une matière d'entretien exceptionnelle : « j'ai lu comment Prowler gère X, j'ai fait différemment parce que… ».

Piste 3 — Restitution (30 min par phase terminée)

Un post court à chaque phase finie. Douze posts sur douze semaines, c'est une présence professionnelle construite sans effort concentré.

Format qui fonctionne : **un problème concret, une explication, une leçon**. Par exemple : « Pourquoi `AuthenticatedUsers` dans une ACL S3 ne veut pas dire ce que vous croyez » — 300 mots, un exemple, une capture. Ce type de post est lu ; « j'ai terminé la phase 4 de mon projet » ne l'est pas.

Le dernier post de la série est l'article long de la phase 10.

7.4 Contribuer à un vrai projet open source

C'est le levier de bagage le plus fort de toute cette liste, et le plus négligé. Faire relire ton code par les mainteneurs d'un outil utilisé par des milliers d'entreprises vaut plusieurs mois de développement solitaire.

Cible naturelle : **Prowler**. Après ta phase 3, tu connais le domaine assez bien pour contribuer.

Démarche, dans cet ordre :

1. Trouve un check AWS **manquant** dans Prowler, ou un check dont la logique a un angle mort (par exemple un CIDR IPv6 non traité, une région non couverte). Ton travail sur cspm-lite te donne exactement cet œil.
2. Ouvre une **issue** décrivant le problème avant d'écrire du code. Attends le retour d'un mainteneur. Cette étape évite le travail jeté et t'apprend la négociation technique.
3. Soumets une **PR** avec test. Réponds aux commentaires de revue sans te braquer.
4. Alternative si tu es intimidé : commence par la **documentation** ou la traduction. Une PR de doc acceptée ouvre la porte, et personne ne te demandera plus tard si c'était du code ou de la doc.

Le bagage réel : une revue de code par un professionnel, l'expérience du workflow open source, et ton nom dans l'historique d'un outil que ton futur employeur utilise peut-être déjà.

7.5 Les quatre extensions multiplicatrices

Après la v1, ces extensions ne sont pas « plus de la même chose » : chacune ouvre un domaine entier.

A. Analyse de chemins d'attaque (attack path analysis)

Construis un graphe des relations IAM : qui peut assumer quel rôle, quel rôle accède à quelle donnée, quelle instance porte quel rôle. Cherche ensuite les chemins entre une entrée exposée (instance publique, utilisateur sans MFA) et une ressource sensible.

- Effort : 15-20 h avec `networkx`.
- Bagage : théorie des graphes appliquée, évaluation des policies IAM en profondeur, et le concept qui a fait la valorisation de Wiz. C'est le sujet le plus intellectuellement dense du domaine.
- Argument d'entretien : « un finding isolé est du bruit ; ce qui compte c'est le chemin entre l'extérieur et la donnée ». Cette phrase te distingue immédiatement.

B. Shift-left : analyse du Terraform avant déploiement

Réutilise tes checks contre les fichiers `.tf` plutôt que contre l'API AWS. Bloque la PR au lieu de constater la faille en production.

- Effort : 12-15 h (parsing HCL avec `python-hcl2`).
- Bagage : tu couvres alors les deux moitiés du marché — sécurité runtime**et** sécurité du code d'infrastructure. Tu comprends pourquoi les deux sont nécessaires et non redondantes (le drift, les ressources créées hors IaC).

C. Detection engineering

Le CSPM est **préventif**. Ajoute le volet **détection** : consomme les événements CloudTrail et écris des règles qui alertent sur un comportement suspect —`ConsoleLogin` sans MFA, `CreateAccessKey` sur le root, `DeleteTrail`, `PutBucketPolicy` rendant un bucket public, énumération massive.

- Effort : 15-20 h.
- Bagage : c'est le métier **blue team / SOC**, un marché entier distinct. Tu apprends le format **Sigma**, la notion de faux positif, le tuning des règles. Et tu peux enfin dire en entretien que tu comprends la différence entre posture et détection.

D. Multi-cloud ou Kubernetes

Deux directions au choix :

- **Azure ou GCP** : ajouter un second provider. L'intérêt principal est la preuve que ton architecture était bonne — si le moteur n'a pas besoin d'être réécrit, ton design de la phase 1 tenait. Le marché est massivement multi-cloud.
- **EKS / Kubernetes** : posture des clusters (RBAC trop permissif, conteneurs privilégiés, absence de NetworkPolicy). Kubernetes est le sujet de sécurité le plus demandé après le cloud lui-même, et le moins bien couvert par les candidats.

Choisis-en **une seule**. Une extension finie vaut mieux que quatre commencées.

7.6 Retenir ce que tu apprends

Sans système, tu auras oublié 70 % des phases 1 à 3 au moment des entretiens. Trois mécanismes suffisent :

Revue espacée. Chaque fiche de la partie 4 est relue à J+1, J+7 et J+30 après sa première lecture. Cinq minutes à chaque fois. Note les dates dans ton journal, ou mets trois rappels de calendrier. C'est peu coûteux et c'est ce qui fait la différence entre « j'ai déjà vu ça » et « je sais l'expliquer ».

Auto-évaluation orale. Toutes les deux semaines, prends la liste 6.4 et réponds **à voix haute**, sans notes, chronomètre à 2 minutes. Les questions que tu rates définissent tes prochaines lectures. Répondre dans sa tête ne compte pas : c'est précisément l'articulation qui manque en entretien.

Enseigner. Explique un concept à un camarade, dans un club, ou en post. La technique de Feynman est fiable : si tu ne peux pas l'expliquer simplement, tu ne le sais pas. C'est aussi le meilleur détecteur de compréhension superficielle.

7.7 Le carnet de preuves

Crée `learning/preuves.md`. Pour chaque compétence revendiquée, trois éléments :

```
### Sécurité IAM et moindre privilège

**Preuve** : PR #14 — policy IAM minimale du scanner, dérivée empiriquement
(lien vers le commit)

**Anecdote (3 phrases)** :
Mon scanner tournait avec ReadOnlyAccess, ce qui était plus large que nécessaire.
J'ai lancé avec une policy vide et ajouté une par une les 23 actions signalées
par les AccessDenied. Résultat : une policy de 23 actions au lieu des ~2000 de
ReadOnlyAccess, et la certitude que l'outil ne peut rien faire d'autre.

**Question à laquelle ça répond** : « Comment appliques-tu le moindre privilège
concrètement ? »
```

Vise 10 à 12 entrées à la fin du projet. Ce fichier est ce que tu relis la veille d'un entretien — pas ton code.

Une règle : **ne revendique jamais une compétence sans entrée dans ce carnet.** Le CV qui promet plus que le carnet ne contient est ce qui fait échouer les entretiens techniques.

7.8 Ce qui détruit le bagage

Les erreurs qui annulent le travail fourni, par ordre de fréquence :

1. **Coller du code sans le comprendre** — qu'il vienne d'un tutoriel, de Stack Overflow ou d'une IA. Test simple : peux-tu réécrire cette fonction de mémoire demain ? Sinon, tu n'as rien appris, tu as seulement avancé. Utilise l'assistance pour *comprendre plus vite*, jamais pour *sauter la compréhension*.
2. **Ne pas finir.** Un projet abandonné en phase 6 vaut moins qu'un projet plus modeste terminé, parce qu'il n'a ni rapport présentable, ni article, ni release.
3. **Perfectionner une phase terminée.** Réécrire le collecteur S3 une troisième fois pendant que les phases 8, 9 et 10 n'existent pas est la forme la plus courante de procrastination technique.

4. **Ne jamais montrer.** Un dépôt privé sans README ne produit aucun bagage social. Le projet doit être public, lisible, et lié depuis ton profil.
5. **Confondre activité et progression.** 200 commits de renommage ne valent pas un check bien compris. Ton indicateur est la ligne « Appris » de ton journal, pas le graphe de contributions.
6. **Oublier `terraform destroy`.** Une facture AWS surprise à 300 € est la principale cause d'abandon de ce type de projet. Budget avec alerte, et destruction systématique en fin de session.

7.9 Après cspm-lite : les 12 mois suivants

Période	Objectif	Pourquoi
Mois 1-3	Finir cspm-lite jusqu'à la release v1.0.0, article publié	Le socle et la preuve
Mois 4	**AWS Solutions Architect – Associate**	Passe les filtres RH ; le projet couvre déjà la moitié du programme
Mois 4-6	**Une** extension de la section 7.5 (recommandé : attack path ou detection engineering)	La profondeur qui te distingue des autres candidats
Mois 5-7	Première contribution acceptée sur Prowler ou équivalent	Validation externe par des professionnels
Mois 7-9	**AWS Security – Specialty**	La certification la plus alignée avec ton profil
Mois 8-12	Candidatures stage/alternance en Cloud Security, DevSecOps ou SOC	Tu arrives avec un projet, une extension, une contribution et deux certifications

Ce parcours te place, à la fin, devant la quasi-totalité des candidats juniors sur ce créneau — non pas parce que tu auras plus travaillé, mais parce que tu auras travaillé de façon cumulative : chaque élément renforce les autres.

7.10 Checklist finale : ai-je vraiment capitalisé ?

À la fin du projet, tu devrais pouvoir cocher :

- [ ] Le dépôt est **public**, avec un README qu'un inconnu comprend en 5 minutes
- [ ] Une **release v1.0.0** taguée
- [ ] Un **rapport HTML d'exemple** consultable sans installer quoi que ce soit
- [ ] Un **article publié** expliquant un contrôle en profondeur
- [ ] Un **carnet de preuves** de 10 entrées minimum
- [ ] Une **présentation de 10 minutes** que tu as enregistrée et réécoutée
- [ ] Les **19 questions de la section 6.4** répondues à l'oral, sans notes
- [ ] Au moins **un scénario d'attaque** exécuté par toi sur ton propre labo
- [ ] Au moins **une contribution** à un projet open source, même documentaire
- [ ] Une **extension** de la section 7.5 commencée ou finie
- [ ] Un **journal** de 30+ entrées, relu au moins une fois
- [ ] La **facture AWS** du projet inférieure à 20 €

Si tu coches dix de ces douze cases, tu ne sors pas de ce projet avec une ligne de CV. Tu en sors avec un domaine.

8. Les exercices — protocole étape par étape

Six exercices progressifs. Tu codes, un jeu de tests te dit immédiatement si c'est juste, puis je relis ton code et je corrige ce que les tests ne voient pas : la lisibilité, la structure, les cas non gérés, les réflexes de sécurité.

Aucune installation nécessaire. Ni `pip`, ni `pytest`, ni `boto3`, ni compte AWS. Uniquement le Python 3.12 déjà présent sur ta machine. Les exercices travaillent sur des dictionnaires qui reproduisent fidèlement les réponses de l'API AWS — ce qui est aussi la bonne façon de construire l'outil, puisque les checks doivent être des fonctions pures.

Les six exercices

#	Dossier	Sujet	Phase	Durée	Difficulté
1	`ex1-modele`	Le modèle de données, enums, dataclasses, score	1	1 h 15	2/5
2	`ex2-registre`	Décorateur, registre, isolation des pannes	1	1 h 30	3/5
3	`ex3-s3-public`	Les 4 chemins d'exposition d'un bucket S3	3	2 h	4/5
4	`ex4-cles-iam`	Credential report, dates, MFA root	3	1 h 30	3/5
5	`ex5-security-groups`	CIDR, IPv6, protocole `-1`, plages de ports	3	2 h	4/5
6	`ex6-rapport`	Agrégation, tri déterministe, rendu Markdown	4	1 h 30	3/5

Environ **10 heures** au total. Fais-les **dans l'ordre** : chacun suppose acquis ce que le précédent t'a appris.

Le protocole, étape par étape

Étape 1 — Voir où tu en es

```
bash learning/exercices/verifier.sh
```

Tout est marqué `A FAIRE` au départ. C'est normal.

Étape 2 — Lire l'énoncé en entier

```
cat learning/exercices/ex1-modele/enonce.md
```

Lis-le **complètement** avant d'écrire une ligne, y compris la section « pièges » — elle décrit les cas que les tests vérifient. Les indices sont repliés : ne les ouvre qu'après avoir vraiment essayé.

Étape 3 — Lire les tests

```
cat learning/exercices/ex1-modele/test_exercice.py
```

Ce n'est pas de la triche : c'est la spécification exécutable. En entreprise, lire les tests avant d'écrire le code est un réflexe recommandé. Les noms de tests français t'indiquent précisément ce qui est attendu.

Étape 4 — Coder

Remplis `exercice.py`. Remplace chaque `raise NotImplementedError` par ton implémentation. Ne modifie ni `test_exercice.py`, ni `commun.py`.

Étape 5 — Faire tourner les tests, en boucle

```
cd learning/exercices/ex1-modele
python3 -m unittest discover -v
```

Ou depuis la racine du projet, avec le détail :

```
bash learning/exercices/verifier.sh ex1
```

Boucle jusqu'au vert. **Ne passe pas à l'exercice suivant tant qu'un test échoue.**

Étape 6 — Demander la correction

C'est l'étape que la plupart des gens sautent, et c'est celle qui fait progresser.

Écris-moi :

> **« corrige mon ex1 »**

Je lis ton `exercice.py` et je te réponds sur cinq axes :

1. **Justesse au-delà des tests** — les cas qu'ils ne couvrent pas
2. **Lisibilité** — nommage, longueur des fonctions, commentaires utiles ou inutiles
3. **Idiomes Python** — ce qu'un relecteur expérimenté écrirait autrement, et pourquoi
4. **Réflexe sécurité** — ce qui manquerait dans un vrai outil de production
5. **Une question ouverte** — pour vérifier que tu as compris, pas seulement réussi

Tu peux aussi demander une correction partielle : *« je bloque sur `policy_allows_anyone`, explique-moi sans me donner le code »*. Je le ferai volontiers — c'est souvent plus utile que la solution.

Étape 7 — Consigner

Une entrée dans `learning/journal/`. Trois lignes suffisent, dont la plus importante : **ce que tu ne savais pas avant**.

Étape 8 — Reporter dans le projet

Une fois l'exercice validé et corrigé, transpose-le dans le vrai code du projet :

Exercice	Destination
1	`src/cspm_lite/engine/models.py`
2	`src/cspm_lite/engine/registry.py`
3	`src/cspm_lite/engine/checks/s3_checks.py`
4	`src/cspm_lite/engine/checks/iam_checks.py`
5	`src/cspm_lite/engine/checks/net_checks.py`
6	`src/cspm_lite/reporters/markdown_reporter.py`

Une branche, une PR, `Closes #<numéro d'issue>`. Après les six exercices, la moitié du cœur de cspm-lite existe déjà — écrite par toi, testée, et relue.

Règles

Ne me demande pas la solution avant d'avoir essayé. Tu as le droit de bloquer 30 minutes ; c'est même là que l'apprentissage se produit. Après 30 minutes réelles sur le même point, demande une explication — pas le code.

Ne modifie jamais les tests. Si un test te paraît faux, dis-le-moi : soit tu as trouvé une vraie erreur (ça arrive, et c'est un bon signe), soit l'écart entre ta compréhension et le test est exactement ce qu'il faut creuser.

Fais-les dans l'ordre. L'exercice 5 suppose que tu as intégré la structure de finding vue en 3 et 4.

Que faire quand un test échoue

Le message d'`unittest` te donne le fichier, la ligne et la valeur attendue. Trois réflexes, dans cet ordre :

1. **Lis le nom du test.** `test_authenticatedusers` t'indique le concept en jeu, pas seulement la ligne fautive.
2. **Reproduis en isolation.** Ouvre `python3` et appelle ta fonction avec l'entrée du test. Regarde ce qu'elle renvoie réellement.
3. **Relis la section de l'énoncé** correspondant à cette fonction. Neuf fois sur dix, la règle exacte y est écrite et tu l'as lue trop vite.

Pour ne lancer qu'un seul test :

```
cd learning/exercices/ex3-s3-public
python3 -m unittest test_exercice.TestPolicy.test_statement_dictionnaire_unique -v
```

8. Présenter le projet en entretien

8.1 La règle d'or

Ne présente que ce qui existe. Un recruteur technique creuse toujours. Dire « j'ai 30 checks » alors que tu en as 12 se détecte en une question, et tu perds tout le crédit du reste — y compris ce que tu as vraiment fait.

La bonne formule quand une partie n'est pas finie : *« la collecte et les checks S3 et IAM sont en place, le volet réseau est en cours, le labo Terraform est ma prochaine étape »*. Un projet en cours et lucide vaut mieux qu'un projet fini et menti.

8.2 Le pitch de 30 secondes

Pour la question « parlez-moi de vous » ou en salon.

> J'ai construit **cspm-lite**, un scanner de posture de sécurité pour AWS. Il se > connecte en lecture seule à un compte, inventorie les ressources, et évalue une > trentaine de contrôles du CIS Benchmark — bucket S3 public, clés IAM trop > anciennes, ports sensibles exposés, CloudTrail désactivé. La sortie va d'un rapport > HTML jusqu'à un export vers AWS Security Hub. Ce que j'y ai vraiment appris, c'est > le risque derrière chaque configuration, pas juste comment appeler l'API.

8.3 La version 2 minutes

La structure : **Problème ? Approche ? Architecture ? Validation ? Apprentissage**. C'est la réponse standard à « présentez-moi un projet dont vous êtes fier ».

> **Le problème.** La majorité des brèches cloud ne sont pas des exploits > sophistiqués, ce sont des erreurs de configuration : Capital One, Accenture, > Verizon. C'est le modèle de responsabilité partagée — AWS sécurise le cloud, le > client sécurise dans le cloud. Je voulais comprendre ça de l'intérieur, donc plutôt > que de lire de la documentation, j'ai reconstruit un outil du type de Prowler. > > **L'architecture.** Il y a quatre couches. Une couche d'authentification qui gère > les profils et l'`AssumeRole`. Des collecteurs qui interrogent S3, IAM, EC2 et > CloudTrail et produisent un inventaire normalisé. Un moteur de règles où chaque > check est une fonction décorée, enregistrée dans un registre. Et des reporters : > JSON, Markdown pour les commentaires de PR, HTML, et ASFF pour Security Hub. > > **Le choix dont je suis le plus content**, c'est la séparation stricte entre > collecte et évaluation. Les collecteurs ne savent rien de la sécurité, les checks ne > savent rien de boto3. Ça me donne deux choses : je teste tous mes checks sans compte > AWS, avec des inventaires fabriqués à la main ; et ajouter un contrôle, c'est écrire > une fonction, pas modifier le moteur. > > **La validation.** Pour vérifier que l'outil détecte vraiment quelque chose, j'ai > écrit un labo Terraform volontairement vulnérable — bucket public, utilisateur avec > AdministratorAccess, port 22 ouvert sur le monde. Je déploie, je scanne, je vérifie > que les findings attendus sortent, je détruis. > > **Ce que j'en retire.** Je ne détecte pas « un bucket public », je sais ce qu'un > attaquant en fait. Et j'ai compris qu'un finding isolé ne vaut rien : ce qui compte, > c'est le chemin entre l'extérieur et la donnée.

Cette dernière phrase est celle qui fait la différence. Garde-la pour la fin.

8.4 Le tableau blanc, en 90 secondes

Si on te demande de dessiner. Dessine de gauche à droite, en parlant :

```
  AWS API  ->  auth  ->  collectors  ->  inventaire  ->  checks  ->  findings  ->  reporters
                          (boto3)        normalise      (purs)                   json/md/html
/asff
```

Les trois phrases à dire pendant que tu dessines :

1. « Les collecteurs sont la seule partie qui parle à AWS. Pagination, multi-région en parallèle, gestion du throttling. »
2. « L'inventaire normalisé est la frontière. Tout ce qui est à droite est du code pur, testable sans réseau. »
3. « Les reporters ne connaissent que le modèle de findings. J'en ai quatre, et en ajouter un ne touche à rien d'autre. »

Si on te pousse : *« et si je te demande d'ajouter Azure ? »* ? « j'écris un collecteur Azure qui produit le même modèle de `Resource`. Le moteur et les reporters ne bougent pas. Les checks, eux, sont spécifiques au provider — c'est la limite de mon abstraction, et je l'assume. »

8.5 Les cinq histoires à avoir prêtes

Une histoire = situation, ce que tu as fait, ce que tu as appris. 60 à 90 secondes chacune. Ce sont elles qu'on retient, pas la liste de technologies.

1. Le moindre privilège, dérivé empiriquement. Mon scanner tournait avec `ReadOnlyAccess`, bien plus large que nécessaire. J'ai relancé avec une policy vide et ajouté une par une chaque action refusée par un `AccessDenied`. Résultat : une policy explicite au lieu d'une policy managée fourre-tout, et la certitude que l'outil ne peut rien faire d'autre que ce qu'il annonce. ? *Répond à : « comment appliques-tu le moindre privilège concrètement ? »*

2. Le piège `AuthenticatedUsers`. Dans une ACL S3, ce groupe ne veut pas dire « mes utilisateurs » : il désigne n'importe quel titulaire d'un compte AWS dans le monde. Un bucket avec cette ACL est public, avec une étape de plus. Je l'ai découvert en écrivant le check, et c'est typiquement le genre d'erreur qu'un développeur expérimenté commet. ? *Répond à : « raconte-moi un truc contre-intuitif que tu as appris. »*

3. L'isolation des pannes. Un check qui plante ne doit jamais interrompre les 29 autres. Mon moteur capture l'exception, produit un finding `NOT_AVAILABLE` avec le message d'erreur, et continue. Sinon, un bucket supprimé pendant le scan rend l'outil inutilisable en production. ? *Répond à : « comment penses-tu la robustesse ? »*

4. IPv6 et le protocole `-1`. Sur les security groups, deux erreurs sont quasi systématiques dans les implémentations amateurs : oublier `::/0`, et mal gérer `IpProtocol: "-1"` qui ouvre tous les ports sans avoir de champ `FromPort`. J'ai écrit les tests pour ces deux cas avant le code. ? *Répond à : « donne-moi un cas limite que tu as géré. »*

5. Le labo vulnérable. J'ai construit en Terraform l'infrastructure que mon outil est censé détecter, puis j'ai exécuté le scénario d'attaque moi-même. Attaquer et défendre la même infrastructure change la façon d'écrire une détection. ? *Répond à : « comment sais-tu que ton outil marche ? »*

8.6 Les questions de relance et comment y répondre

« En quoi c'est différent de Prowler ? » Ce n'est pas différent, et c'était volontaire. Prowler est mature, je ne prétends pas faire mieux. Mon objectif était de reconstruire l'architecture pour comprendre les décisions qu'elle contient. J'ai d'ailleurs lu leur code et j'ai fait certains choix différemment — par exemple, eux chargent les checks dynamiquement depuis le système de fichiers, moi j'utilise un registre explicite, ce qui est plus simple à tracer mais moins souple.

« Quelles sont les limites de ton outil ? » La meilleure question qu'on puisse te poser. Réponse honnête et structurée :

- pas d'**analyse de chemin d'attaque** — je sors des findings isolés, pas le chemin entre une entrée exposée et une donnée sensible ;
- pas de **contexte métier**, donc pas de vraie priorisation : un bucket public de logs publics est un faux positif que je n'ai aucun moyen de distinguer ;
- c'est un **instantané**, pas de la détection temps réel — le volet CloudTrail serait la suite logique ;
- **un seul provider**, et mes checks ne sont pas transposables tels quels.

« Combien de temps ça t'a pris ? » Ne minimise pas. « Environ 80 heures sur trois mois, en parallèle des cours. » Un chiffre honnête montre que tu sais estimer.

« Pourquoi des threads et pas asyncio ? » Le scan est *I/O bound* : on attend l'API AWS. Les threads suffisent, le GIL n'est pas un problème puisqu'on attend du réseau. `asyncio` aurait demandé `aioboto3` et compliqué le code pour un gain nul à cette échelle — 17 régions, pas 10 000 requêtes.

« Comment testes-tu du code qui appelle AWS ? » Deux niveaux. Les checks sont des fonctions pures, je leur injecte des inventaires fabriqués — c'est instantané et ça couvre tous les cas limites. Les collecteurs, je les teste avec `moto`, qui simule les services AWS en mémoire. Le point important, c'est que mon architecture rend le premier niveau possible : c'est le retour sur investissement de la séparation collecte/évaluation.

« Qu'est-ce que tu ferais différemment ? » J'aurais figé le modèle de `Finding` plus tôt : je l'ai fait évoluer trois fois et chaque changement m'a coûté du travail dans les quatre reporters. Et j'aurais écrit le labo Terraform avant les checks plutôt qu'après — j'aurais validé au fur et à mesure au lieu de découvrir des écarts à la fin.

« Tu l'utiliserais en production ? » Non, pas tel quel. Il n'a pas la couverture de Prowler, pas de gestion multi-comptes via AWS Organizations, et je n'ai pas de retour terrain sur les faux positifs. En revanche, il m'a donné les bases pour utiliser et comprendre un vrai CSPM, y compris pour savoir quoi contester dans ses résultats.

8.7 Adapter selon l'interlocuteur

Interlocuteur	Ce qu'il veut entendre	Durée	À éviter
RH	Le problème résolu, en français courant, et un chiffre	45 s	Tout jargon technique
Ingénieur sécurité	Les contrôles, le risque réel, ton scénario d'attaque	3-5 min	Parler surtout de Python
Lead dev / architecte	La séparation des couches, la testabilité, les compromis	3-5 min	Parler surtout de sécurité
Manager / CTO	Pourquoi tu l'as fait, ce que tu mesures, tes limites	2 min	Les détails d'implémentation

Pour le RH, la version courte : *« un outil qui vérifie automatiquement qu'un compte cloud est configuré sans faille de sécurité, en s'appuyant sur un référentiel international. C'est le type d'outil que les entreprises utilisent pour éviter les fuites de données qu'on voit dans la presse. »*

8.8 Les pièges à éviter

1. **« C'est juste un projet perso »** — ne dévalorise jamais ton propre travail en ouverture. Dis ce que c'est, pas ce que ce n'est pas.
2. **Réciter.** Un pitch appris par cœur s'entend et bloque le dialogue. Retiens la *structure* (problème ? architecture ? validation ? apprentissage), pas les phrases.
3. **Le name-dropping.** Énumérer « Python, boto3, Terraform, Docker, GitHub Actions, pytest, moto » ne dit rien. Une décision technique expliquée vaut dix technologies citées.
4. **Ne pas savoir dire ce qui ne marche pas.** Un candidat sans limites à énoncer n'a pas assez creusé. Prépare tes limites autant que tes réussites.
5. **Le nombre de checks comme argument.** Personne n'est impressionné par 30 checks. Un check expliqué en profondeur, avec l'incident réel qu'il aurait évité, oui.
6. **Oublier la question retour.** Termine par une question : *« vous utilisez quoi pour la posture cloud, et sur combien de comptes ? »* Tu passes de candidat interrogé à interlocuteur.

8.9 La veille de l'entretien

- Relis ton **carnet de preuves** (section 7.7), pas ton code.
- Redis le pitch de 2 minutes **à voix haute**, une fois, chronométré.
- Redessine l'architecture sur une feuille, sans regarder.
- Prépare **une** anecdote parmi les cinq, celle qui colle le mieux au poste.

- Vérifie que ton dépôt est accessible, que le README s'affiche bien, et que le lien vers le rapport HTML d'exemple fonctionne.
- Prépare tes deux questions à leur poser.